Resume

Latex PDFs

You can download the PDF version of my resume, created with LaTeX, here or the dark mode version.

You can also find the Chinese version here or the dark mode version.

Profile

- Male, born in 1995, Chinese citizen, resides in Guangzhou. Achieved top 300 in Guangdong NOIP 2011 and solved around 1000 algorithm problems online.
- Studied for one year at Beijing Forestry University before dropping out. Pursuing an associate degree through self-taught education, with nine courses completed.
- Currently at TEKsystems (outsourced to HSBC Bank). Previous experience includes LeanCloud, Beijing Square Root, Beyondsoft (outsourced to DBS Bank), and Farben Information (outsourced to HSBC Bank).
- Accumulated 11 years of professional experience, including 8 years in corporate roles and 3 years as a freelancer. Specializes in 2 years of mobile (Android, iOS) engineering, 8 years of full-stack engineering (7 years backend, 1 year frontend), and 1 year in machine learning/big data.
- Good understanding of HTML, CSS, Browser APIs, JavaScript, Vue.js, React, Angular, and iOS/Android development. Familiar with Java, Spring, Spring Boot, MySQL, Redis, APIs, networking, containers, and distributed systems. Familiar with logging, database, and microservice modules on Alibaba Cloud, AWS, and Azure.
- Native in Chinese, proficient in English (IELTS 6). Read over 320 books spanning textbooks, personal development, business, technology and history. Read around 2000 comprehensive, long AI answers in the past year.
- Developed 10 open-source GitHub projects as a main developer, each with over 500 commits available at https://github.com/lzwjava. Detailed software portfolio available at https://lzwjava.github.io/pages/portfolio-en.
- Generous knowledge sharer with AI assistance to help fix grammar, having around 480 original posts on the blog attracting around 60,000 page views a year, and who documents a lot in the corporate environment too.
- Self-taught researcher with three academic-style papers on reversing myopia and natural vision restoration, inspired by the works of Yin Wang and Todd Becker, backed by three years of experimentation.
- Engineer with an entrepreneurial spirit and a product mindset, using 3 mobile phones and one thousand of apps and games to play with. Work as an autonomous AI agent, proficient with extensive AI tools, and becoming better with comprehensive logging and testing.
- Life hacker in every aspect of life, with one thousand of innovative practices—from bringing a second mobile device to help my daughter learn English in China, to using a car vent to dry hair; air fryer lover; owns hundreds of small tools and gadgets like an Electric/Magnetic Fields meter, etc.
- Travel lover and road trip master. Been to the USA twice, as well as Hong Kong and Macao, and half parts of China. Taken English courses online with 60 Filipino English teachers, and some African or European teachers too.

Work Experience

Meitai Technology Service (Shanghai) Co., Ltd. Guangzhou Branch, Backend Engineer, 2025.02 - Present

- Backend Engineer outsourced to HSBC Bank under TEKsystems, developing and optimizing backend systems for the Finance Transformation Platform in HSBC's Enterprise Technology Department.
- Maintained features for financial data processing including import, validation, and export, while enhancing submission and approval workflows for new financial headers. Leveraged expertise in accounting, ledger, and banking systems.

- Managed the full development lifecycle, from local development to UAT testing and production releases. Copilot was heavily utilized for extensive tasks such as analyzing root causes, generating Python scripts, and documenting around 200 technical guides; some of these guides became key references for teammates facing similar challenges.
- Developed innovative banking technical solutions by leveraging AI tools like Copilot, performance profiling with YourKit, and systematic testing while functioning as an autonomous AI agent who analyzed project architecture, collaborated with teams, and continuously improved through comprehensive logging and documentation.
- Tried to implement an AI software agent in a big financial project using AspectJ to gather tons of logs and convert debug states to text, providing context for AI chatbots.
- Worked with technologies such as Java, Spring Boot, IBM Db2, multithreading, Maven, Nexus, Angular, and Python, along with tools including Windows, Control-M, IBM WebSphere Application Server, and Copilot.

Freelancer, 2023.08 - 2025.01

- Analyzed and reimplemented some core parts around 30 example projects about machine learning, from PyTorch, TensorFlow tutorials, Coursera online courses or other open-source projects. And got two course completion certificates from Coursera about Machine Learning Specialization and Deep Learning Specialization.
- Prepared for exams related to obtaining an associate degree, with a focus on subjects such as advanced mathematics, computer organization, linear algebra, etc. Listened to a lot of Japanese songs and watched a lot of Japanese TikTok videos to learn the language.
- Participated as a Full Stack Developer in an AI-powered story bot project using Claude's API for personalized stories. The bot supports prompt setting and includes a management page for configuration. Developed with Python, Flask, React, and Nginx, and deployed on AWS. Used Prometheus for monitoring, ELK stack for log management, and ChatGPT-4 for programming assistance.
- Maintained a technical blog with 431 original blog posts. Utilized large language models to translate content into 9 languages, provide text-to-speech audio, support PDF and EPUB formats, integrate with GitHub workflows, and use LaTeX for writing papers or creating a resume. Other technologies employed include Python, Jekyll, Deepseek, and Mistral. The website is https://lzwjava.github.io.
- Conducted experiments with llama.cpp, embeddings, rerankers, Retrieval-Augmented Generation, and the MMLU benchmark. Explored search engine provider integrations such as Jina AI, Tavily AI Search API, and ElevenLabs API.

Shenzhen Farben Information Technology Co., Backend Engineer, 2022.11 - 2023.07

- Farben is a leading software technology service provider in China. HSBC Bank is one of the largest banking and financial services institutions in the world. PayMe is an HSBC mobile payment service catering to Hong Kong residents.
- Contracting through Farben, assigned to work for HSBC Bank and contributed to the PayMe project. Participated in the backend development for the Auto Top Up feature, which automatically tops up funds from the user's credit or debit card when their balance falls below a certain amount. Monitored post-payment events from Azure EventHub and checked for top-ups whenever a user set up their auto top-up configuration. Used dedicated object-oriented programming to handle cases elegantly and applied aspect-oriented programming to audit logs for changes in the auto top-up table.
- Actively contributed to cloud migration following participation in the company's AWS training. Refactored APIs to utilize request header-based routing, ensured secure access and database configurations, and participated in deploying microservices to the new cloud infrastructure.
- Utilized a strong tech stack including Java, Spring, and Kafka, along with Azure, Azure DevOps, and AWS for cloud services and continuous integration.

Beyondsoft Guangzhou Co., Ltd., Backend Engineer, 2021.12 - 2022.11

- Beyondsoft Consulting is a leading business IT and consulting company. DBS Bank is the largest bank in Southeast Asia by assets and among the largest banks in Asia. Contracting with Beyondsoft, assigned to work for DBS Bank and contributed to DBS Client Connect and DBS DigiBank CN projects.
- In the DBS Client Connect project, contributed to the development of the equity trading microservice. Responsibilities included creating functionalities for equities display, customer display, pre-trade checks, and placing actual equity trade orders. Integrated Avaloq APIs to enhance the underlying infrastructure and improved the user experience by implementing an edit distance algorithm to refine stock code searches.
- In the DBS DigiBank CN project, actively contributed to the development of several microservices handling mutual funds management, structured investment products, investment portfolios, and transaction lists. Assisted with performance tests by analyzing logs from Pivotal Cloud Foundry to generate reports on the QPS of the microservices. Developed a tool to automate test generation for the testing tool Karate, streamlining the testing process and improving test coverage.
- My first English-speaking job in an international team at a big company, with Singaporean and Indian colleagues, led me to use LinkedIn for global connections, learning, and exposure to international vibes. It revealed mindset differences beyond language among Chinese, Singaporeans, and Indians. This life-changing shift transformed me, starting with English, by changing my info feed to mostly English—gaining new worldviews and perspectives from those cultures.
- Leveraged cloud technologies and modern frameworks, employing automation best practices with tools such as Java, Spring Cloud, Jira, Confluence, Jenkins, Pivotal Cloud Foundry, and Kibana, and utilizing BDD and TDD methodologies.

Freelancer, 2020.01 - 2021.11

- Authored and published technical blogs to share knowledge online, improved English proficiency through engaging with Netflix and literature, and enhanced problem-solving skills by tackling around 500 algorithm problems and participating in Codeforces competitions. Gained a little hands-on experience with big data and cloud-native technologies by exploring introductory tutorials and running examples for Spark, Hadoop, Kubernetes, and Docker.
- Completed various software projects as a freelancer, LED Sign Website Development, Enterprise WeChat Integration for ShowMeBug, Web Scraper for Trade Data Collection, eBook tool mathjax2mobi.
- LED Sign Website Development (lvchensign.com): Developed a website for an LED sign manufacturing company using Bootstrap, HTML and JavaScript. Implemented features for showcasing products.
- Enterprise WeChat Integration for ShowMeBug: Contributed to the integration of ShowMeBug with Enterprise WeChat, enabling seamless access to technical interview tools within the Enterprise WeChat ecosystem. Utilized technologies like Ruby, Ruby On Rails, PostgreSQL and the WeChat SDK to create a smooth user experience for interviewers and candidates.
- Web Scraper for Trade Data Collection: Developed a web scraper using Python and Selenium to collect trade data for a non-woven fabric company. Automated data extraction and page navigation, processed and stored the data in a SQLite database, and generated reports for business analysis.
- mathjax2mobi: a tool designed to convert HTML content with MathJax equations into eBook-friendly formats. By transforming LaTeX-based MathJax equations into SVG images, it ensures compatibility with eBook formats like MOBI. Technologies used include Python, BeautifulSoup, and Selenium.

Beijing Square Root Inc, Founder & Full Stack Engineer, 2016.07 - 2019.12

- Beijing Square Root Inc. ran two businesses in 3.5 years. From July 2016 to September 2017, it rolled out and operated Fun Live, a knowledge live broadcast platform. From January 2018 to December 2019, it transformed into a software consulting business.
- With Fun Live, users could participate in various knowledge lectures such as programming or design. Users could pay fees to attend live sessions or reward lecturers. The lecturer used the OBS tool to push live streams to the server. Users could participate in the lecture in real-time or see the playback later. The platform integrated seamlessly with WeChat for notifications. Hosted around 80 lectures, gained 30,000 users and millions of page views.
- As a solo entrepreneur, I coded during the day and gained users at night; everything was direct with minimal bureaucracy. This was my peak performance. I registered the company, met with investors to secure an investment of half a million CNY, handled equity changes, attended other engineering conferences to reach out to potential lecturers, and built 10 WeChat groups with around 500 people each. The highs and lows were intense and exhilarating.
- After getting investment, I hired 9 employees but had to let them go after two months, which contributed to one of my most painful mistakes: not having a long-term plan, wanting to impress others, and naively trying to train my management skills. The good news is that by hiring part-time engineers to do 50 small software projects together, we earned half a million back and gave it back to the investor when shutting down the company.
- From an engineering perspective, I performed most of the software development, resulting in around 2,000 code commits across 3 web client apps and 1 server. They were open sourced on GitHub@lzwjava some years later. The project leveraged PHP, Vue, HTML, CodeIgniter, MySQL, Redis, LeanCloud, Alibaba Cloud, WeChat SDK, SRS

(Simple Realtime Server), WeChat MiniProgram, Fabric, Linux, and Nginx. I also did some design work in Sketch.

- During the software consulting business period, completed 50 small software projects for customers, including websites, games, and apps. The revenue was around 3 million RMB and the profit was around 700 thousand RMB. Responsible for project negotiation, team management, and some software development. Notable projects are shown below.
- MianbaoLive: Led a comprehensive backend refactoring effort for MianbaoLive, a one-stop content monetization and social economy platform. Optimized the performance, stability, and user experience of the entire tech stack. Previously used ThinkPHP, Node.js, and Go, and rewrote everything in Laravel for server-side development. The platform included modules for courses, users, content, user attendance, payments, and distribution sales. Partnered with Himalaya, a top audio platform in China, and synchronized content between the platforms.
- Jiangsu TV's Super Brain Program WeChat Mini Program: Responsible for all backend development and half of the frontend development for the Super Brain program's WeChat Mini Program. Engaged viewers with interactive puzzle games, enabling them to compete and rank to become the "Super Brain."Used the WeChat Mini Program framework and Wepy (Vue.js) to create game components and ranking pages. Integrated RESTful APIs to fetch game data and user information. Performed extensive performance tuning to ensure the system could handle high concurrency, leveraging caching technologies such as Redis.
- Chongding Conference: Spearheaded full-stack engineering for Chongding Conference, a mobile trivia app in China resembling HQ Trivia. Designed and implemented services and APIs to handle live trivia events, user management, and real-time question and answer sessions. Utilized Java and Spring for the backend, with Redis and Kafka for caching and message queuing, Zookeeper for service coordination, and Socket.IO for real-time interactions. Developed the admin panel to help operators control the game. The app supported live streaming, real-time interaction, and robust performance under high traffic conditions. Contributed to technical discussions for using SEI (Supplemental Enhancement Information) to synchronize the timestamp of the live streaming with the trivia game interactions.

Beijing Dami Entertainment Co., Cofounder & Full Stack Engineer, 2015.11 - 2016.7

- Beijing Dami Entertainment Co. was a company founded by 6 Internet enthusiasts, including me. Launched and operated the platform CodeReview, a professional platform for code review, communication, and sharing. Gained around 3000 users.
- The platform included functionalities such as user management, code submission and review processes, notification systems, payment integration, and event and workshop management. Engineers could submit their code for expert review to improve its quality, and experts received fees for their reviews. The platform also offered workshops and events open to users. Responsible for the development of the backend and half of the frontend. Utilized a strong tech stack including PHP, Vue, CodeIgniter, Alibaba Cloud, and Ping++.
- My first entrepreneurial attempt was with 5 cofounders, who helped me a lot with my own startup and provided inspiration for me to become better. We were from big companies or cutting-edge startups in Beijing. Only later in life could I appreciate the entrepreneurial spirit in Beijing and the greater chance to meet people who dream big and pursue excellence instead of doing mediocre jobs. The atmosphere in Beijing at that time was quite unique and exciting.

Delicious Bookmarks (Beijing) Information Technology Co., Ltd., Software Engineer, 2014.07 - 2015.11

- Delicious Bookmarks is a leading cloud computing provider in China, known as LeanCloud. It offers a complete set of cloud services including object storage, file storage, web hosting, containers, instant messaging, push notifications, SMS, and game backends. The company serves hundreds of thousands of developer users.
- Participated in the development of LeanCloud Objective-C SDK and Java SDK. Responsible for the development of LeanChat iOS client and Android client, a chat application designed to demonstrate the instant messaging SDK. Additionally, worked on various frontend projects. Employed a robust tech stack, leveraging modern tools such as iOS SDK, Android SDK, Cocoapods, Xcode, Android Studio, and the Angular framework.
- This was my first job. In the early days of my university life, I participated in the 2013 China Software Developers Conference hosted by CSDN and met the head of engineering at LeanCloud. As a result, I later interned at this company and dropped out to become a full-time employee. The company was sold to the Hong Kong public company TapTap around 2021, and I received compensation for my vested options. The team I met there is still one of the best I have worked with.

Educational Background

Guangdong University of Foreign Studies□Computer Application, Associate Degree, 2022.10 - Present

- National Higher Education Self-study Examination. Passed 9 courses as of today, Introduction to Operating Systems, Introduction to Data Structures, Advanced Language Programming (I), English (I), Advanced Mathematics, Database and Applications, Computer Application Technology, Introduction to Law and Computer Organization and Architecture.
- Currently preparing for 7 courses such as Computer Networks, Basic Electronic Technology (III), Microcomputer and Interface Technology, Mao Zedong Thought and Theory of Socialism with Chinese Characteristics, Ideological and Moral Cultivation and Rule of Law, College Chinese and Linear Algebra.

Beijing Forestry University, Digital Media Arts, Bachelor(Dropout), 2013.09 - 2014.06

- Passed 13 courses, like Computer Introduction, Fundamentals of Programming, Programming Fundamentals Lab, College English, Advanced Mathematics, Principles and Techniques of Digital Imaging, Two-Dimensional Graphics Technology.
- Built a campus assistant Android application “Bjfu Assistant”that integrates the functions of checking grades, and alumni forums. It has been downloaded and used by thousands of users on campus. And I built an Android App for learning English by watching movie clips, and won the third prize in a hackathon.
- Ranked in the top 1 in my school’s ACM/ICPC monthly training competition among around 100 top students in the university, including higher grade one.
- Awarded third prize in the “Youth, Health, New Media”digital works creativity contest organized by UNESCO and the Communication University of China for the digital work “Mom Shy To Say”.

Guangzhou Yuyan Middle School, Science, Middle School Diploma, 2007.07 - 2013.06

- Ranked in the top 300 out of 100k students in the Guangzhou High School Entrance exam.

- Ranked in the top 300 in the 2011 Guangdong NOIP(National Olympiad in Informatics in Provinces).

Projects

Corporate Projects

- HSBC Finance Transformation Platform (2025.02 - Present) Participated in the backend development. Used tech stacks like Java, Spring, Python, IBM WebSphere and Copilot.
- AI-powered Story Bot (2024.5 - 2024.7) Participated in the full stack development. Used tech stacks like Python, Flask, React, AWS, and Claude.
- PayMe from HSBC (2022.11 - 2023.07) Participated in the backend development of the auto top-up feature and cloud migration. Used tech stacks like Java, Spring Cloud, Azure, and AWS.
- DBS Digibank CN (2022.7 - 2022.11) Participated in the backend development of mutual funds. Used tech stacks like Java, Spring Cloud, and Pivotal Cloud Foundry.
- DBS Client Connect (2021.12 - 2022.6) Participated in the backend development of equity trading. Used tech stacks like Java, Spring Cloud, and Pivotal Cloud Foundry.
- ShowMeBug (2021.7 - 2021.9) Responsible for the integration between the platform and Enterprise WeChat, including login and easy interview entrance. Used tech stacks like Ruby on Rails, PostgreSQL, and WeChat SDK.
- Square Root Inc Projects (2018.1 - 2019.12) Participated 50 miscellaneous projects including websites, games, apps, and WeChat mini programs. Responsible for team management, contract negotiation, project management, client communications, and some software development.
- Super Brain TV Program WeChat Mini Program (2018.2 - 2018.3) Responsible for the backend and frontend development. Used tech stacks like Java, Spring, JavaScript, HTML, and AliCloud.
- Chongding Conference (2017.9 - 2018.1) Responsible for some backend development and frontend development of the admin panel for a live-streaming trivia app. Used tech stacks like Java, Spring, WebSocket, and Netty.
- Mianbao Live (2017.9 - 2018.1) Responsible for refactoring the backend of a one-stop content monetization platform. Used tech stacks like PHP, TypeScript, Laravel, and Go.
- Fun Live (2016.6 - 2017.9) Responsible for most code development of a knowledge live broadcast platform. Used tech stacks like PHP, JavaScript, HTML, AliCloud, and RTMP Streaming.
- LeanCloud SDK and LeanChat (2014.07 - 2015.10) Responsible for some Objective-C and Java SDK development, and all development of their demo app LeanChat.

Open Source & Blog & Codeforces (2013-09 to Present)

- Explore my open-source projects, corporate initiatives, and personal side projects on GitHub: https://github.com/lzwjava
- algorithm-solutions: Solutions to algorithm problems, 2466 commits, Java
- lzwjava.github.io: My blog, 500 commits, English & Chinese
- live-server: Backend of a knowledge live broadcast platform, 660 commits, PHP
- live-mobile-web: Mobile frontend of a knowledge live broadcast platform, 528 commits, Vue & JavaScript
- live-web: Desktop frontend of a knowledge live broadcast platform, 140 commits, Vue
- live-wxapp: WeChat Miniprogram of a knowledge live broadcast platform, 63 commits, JavaScript

- code-review-server: Backend of a professional platform for code review, 275 commits, PHP
- code-review-web: Frontend of a professional platform for code review, 302 commits, Vue & JavaScript
- weimg-server: Backend of an image community, 35 commits, PHP
- weimg-ios: iOS application of an image community, 77 commits, Objective-C
- leanchat-ios: iOS chat application, 556 commits, Objective-C
- leanchat-android: Android chat application, 412 commits, Java
- curiosity-courses: A programming course, 215 commits, Chinese
- LZAlbum: An iOS application similar to WeChat moments, 47 commits, Objective-C
- TabsKiller: A Chrome plugin to help close browser tabs, 19 commits, JavaScript
- Reveal-In-GitHub: An Xcode plugin, 57 commits, Objective-C
- LvchenSign: Website for an advertising signage company, 40 commits, JavaScript
- feynman-lectures-mobi: Tool to convert LaTeX to SVG for creating mobi ebooks, 47 commits, Python
- flower-recognition: Android application for flower recognition, 37 commits, Java
- Creak: A simple HTML parser, 49 commits, Swift

Skills

I have gained a lot of skills since I started programming in 2008. In terms of usage time, the high-level description is as follows. It is different from working experience. I use it to show relative familiarity with my skills.

- 5 or more years of using experience: Chinese, English, Writing, Java, Algorithm, Test-driven Development, Cloud Computing, DevOps, Backend Development
- 3 to 4 years of using experience: Cantonese, iOS Development, Android Development
- 1 to 2 years of using experience: Product Management, Machine Learning, Frontend Development
- less than 1 year of using experience: Japanese

Tools

Below are detailed tools that help me to accumulate the above skills, plus other interesting tools. I am still using some of them and can learn for a few days to use the rest of them again.

- Natural Language: English, Chinese, Cantonese, Hakka, Japanese
- Operating System: Linux, Windows, MacOS, Android, iOS, OpenWRT
- Programming Language: PHP, Java, Objective-C, Swift, Ruby, HTML, C, C++, JavaScript, Lisp, Clojure, TypeScript, Node.js, Go, Python, Pascal
- Domain-specific language: JSON, CSS, SQL, NoSQL, Kusto Query Language, Kibana Query Language
- Database: MySQL, SQLite, PostgreSQL, Microsoft SQL Server, Amazon RDS, MongoDB, IBM Db2, Navicat
- Database Client: SQuirreL SQL Client, DBeaver, MySQL Workbench, Sequel Pro, Navicat, Eclipse Data Source Explorer
- Frontend: Vue, React, Angular, jQuery, Hexo, Jekyll, Stylus, Sass, Chrome, Safari
- Framework: RTMP streaming, HTTP/IP, Spring, Spring Boot, Hibernate, Netty, CodeIgniter, Laravel, Flask, Express, Socket.IO, iOS SDK, Android SDK, Flutter

- Big Data: Redis, Kafka, Spark, Flink, ZooKeeper
- Math: Calculus, Linear Algebra, Discrete mathematics, Latex
- IDE: Emacs, IntelliJ IDEA, Visual Studio Code, Xcode, Android Studio, Eclipse, PyCharm, PhpStorm, Sublime Text, Atom, WebStorm, Delphi, Turbo Pascal
- Terminal: Powershell, Command Prompt, iTerm2, GNOME Terminal, Terminal (macOS)
- Development: NPM, Webpack, Git, HomeBrew, Gradle, Maven, Sketch, Cocopods, WordPress, YourKit Java Profiler
- CI/CD: Travis CI, GitHub Actions, Jenkins, Azure DevOps, Control-M
- Monitoring: SonarQube, Prometheus, Elasticsearch, Logstash, Kibana, Nginx, Docker, Kubernetes
- Machine Learning: TensorFlow, PyTorch, Theano, Anaconda, Jupyter Notebook, CUDA, Numpy, Pandas
- Cloud: GitHub, GitLab, Digital Ocean, Vultr, AWS, Azure, Google Cloud Platform, Linode, Alibaba Cloud, Qiniu Cloud, Tencent Cloud, LeanCloud, Firebase, Cloudflare, GoDaddy, Agora, Vercel
- Application Runtime: IBM WebSphere Liberty Server, Pivotal Cloud Foundry, PM2
- Project: Jira, Confluence, ServiceNow, Trello, Microsoft To Do, Tower.im,
- Large Language Model: OpenAI, Claude, MiniMax, Amazon Bedrock, Copilot
- Linux: nmap, grep, awk, ffmpeg, htop, df, lsof, strace, tcpdump, iostat, iptables, rsync, journalctl, systemctl, curl, tmux, traceroute, emacs, diff
- Testing: Postman, Selenium, Karate, JUnit, Mockito, Apache JMeter, Apache ab
- Hardware: Arduino UNO, Raspberry Pi Pico

Domain Knowledge

Below is a summary of my expertise and knowledge domains. While I have a solid understanding of some topics and a more basic familiarity with others, I would be happy to share more in an interview if you are interested in any specific areas.

- Industry Knowledge: Banking and Payments, Education and E-learning Platforms, Social Media and Content Platforms, User and Account Management, Notification Systems, Data and Analytics, Mobile Applications, Payment Systems, Transaction Management, Social Media Integration, Video Management, Reward Systems, Live View Tracking, Application Management, Mutual Funds and Equity Trading, Browser-Based Programming Environments.
- Computer Science Fundamentals: Advanced Mathematics, Computer Organization, Operating Systems, Computer Network Technology, Database and Applications, Computer Application Technology, Data Structures and Algorithms, Microcomputer and Interface Technology
- Technology and Development: Multilingual Communication, Cross-Platform Development, Full-Stack Programming, Database Management, Machine Learning and Big Data, Mathematical Proficiency, Development Tools, Machine Learning Implementation, Advanced Linux Usage, Testing and Quality Assurance, API Integration, Open Source Contributions, Technical Writing and Blogging, Cloud Computing Services, Distributed Systems, High Performance Optimization, Realtime Application, RTMP Streaming, Continuous Integration/Continuous Deployment (CI/CD), Containerization and Orchestration, Network Security, Agile Methodologies, Software Architecture, DevOps Practices, Cloud-Native Applications, API Development, Version Control Systems, Serverless Computing, Performance Monitoring, Data Engineering, Security Best Practices, Software Development Lifecycle (SDLC), Technical Mentoring, Project Management.

Selected Blog Posts

Below is a selection of my blog posts, covering personal growth stories, startup journeys, English learning, programming, geek projects, and more. Please visit lzwjava.github.io and search the titles to check them out.

- Building a Scalable Application on Azure
- Comprehensive Guide to Spring Framework
- Build an AI-Powered Story Bot with Flask, React, and ELK
- End-to-End Trace ID Implementation
- Deploying with Pivotal Cloud Foundry
- The Wisdom of Learning
- Zen and the Art of Machine Learning
- How to Build a Computer
- Introduction to Web Programming
- Practical: Scraping Website Content
- Mastering English: On and Off the Job
- The Journey of Learning English
- How to Improve English Proficiency with Large Amounts of Input
- Turning Feynman's Lectures on Physics Webpages into an eBook
- Setting Up Live Streaming with OBS, SRS, and FFmpeg
- Lieyunwang: Post-95 Entrepreneur Founded Knowledge Live Streaming Platform QuZhibo
- Enhancing iOS Development with Automated Testing and Tools
- My Trip to the United States: First Impressions of Silicon Valley
- Advanced Git Operations and Principles
- Exploring WebSocket
- Porting FFmpeg to Android

- A Deep Dive into Custom Drawing in Android

Papers

I'm a self-taught researcher with three academic-style papers on reversing myopia and natural vision restoration, inspired by the works of Yin Wang and Todd Becker, supported by three years of experimentation. In computer science, I am still striving to make similar breakthroughs.

- Li, Zhiwei. (Jun 2023). *Experimental Verification of the Natural Vision Restoration Method.* Zhiwei's Blog. http://lzwjava.github.io/vision-restoration-en.
- Li, Zhiwei. (Jun 2023). *The Discussion Regarding Astigmatism When the Eyeballs Reverse to the Normal Shape.* Zhiwei's Blog. https://lzwjava.github.io/astigmatism-en.
- Li, Zhiwei. (Nov 2024). *Natural Vision Restoration: The Principle of 'Just Barely Clear'.* Zhiwei's Blog. https://lzwjava.github.io/barely-clear-en.

Books Read

I have read over 320 books spanning textbooks, personal development, business, technology and history. Below are key books and resources that have shaped my thinking.

- Beginning Algorithm Contests - Rujia Liu
- Steve Jobs - Walter Isaacson
- Treat Time as a Friend - Xiaolai Li
- The 7 Habits of Highly Effective People - Stephen R. Covey
- Sapiens: A Brief History of Humankind - Yuval Noah Harari
- Computer Systems: A Programmer's Perspective - Randal E. Bryant, David R. O'Hallaron
- Yin Wang's Blog (yinwang.org)
- Paul Graham's Essays (paulgraham.com)

Entrepreneurial Spirit and Product Mindset

- Founded two startups: one with co-founders, and one created by myself. The latter gained 30k users and once achieved revenue of 3 million CNY a year.
- Secured an investment of around half a million CNY in my startup at 22 years old, and returned it to the investor when shutting down.
- Applied to Y Combinator twice. One time, entered the second round with around a 3% acceptance rate, though didn't make it into the batch.
- Like to initiate new ideas in big companies to actually help the project; tried many ways and created proofs-of-concept in local development in some code branches. Presented them to tech leads to move things forward. Some received good feedback.
- Gained a lot of social media influencing practice, including blogging, WeChat Subscription Channel, TikTok, WeChat Videos, Weibo and LinkedIn. Acquired thousands of followers in several accounts or 60k page views a year.

- Used 3 mobile phones at the same time and thousands of apps and games to experiment with, including different kinds of apps from tens of countries.
- As an early user of Blue Sky, I commented on Jeff Dean's first post here and got a like from him; he is Google's Chief Scientist.
- Own hundreds of small tools and gadgets, such as an Electric/Magnetic Fields meter, telescope etc. And bring them while traveling to discover their potential in different scenarios.
- If things or products are cheap, very willing to buy different kinds to compare and understand their nuances.
- Read around 100 books about business, management, and product. Have some connections with people who are experts in these areas.
- As a crypto and US stock investor since 2018, I took a break due to some losses but resumed investing in 2024, using long-term and regular investing practices. Despite the modest amount, I am pleased to have gained a 25% profit from NVIDIA after one year of regular investing as of July 2025, as well as a 30% profit from some of my Bitcoin investments.

Links

- GitHub
- Blog
- Codeforces
- UVa Online Judge
- Kaggle
- LinkedIn

Certificates

- Beijing Forestry University Dropout Certificate
- China Higher Education Official Report (Bachelor Education, Dropout Status)
- IELTS Academics, Score 6
- Machine Learning Specialization Course Completion Certificate, provided by DeepLearning.AI and Standford University
- Deep Learning Specialization Course Completion Certificate, provided by DeepLearning.AI
- Developing on AWS, provided by AWS Training

Expectation

- Expected Positions: Full Stack Engineer / Backend Engineer
- Expected Locations: Guangzhou

Recommended Blog Posts for Engineers

These recommendations are generated by AI based on blog posts from the last year, tailored for a 10-year backend engineer. Update this file by running the script every half year to refresh based on new posts.

Here are the most relevant blog posts for a 10-year experienced backend engineer, focusing on technical, programming, and engineering-related topics.

AI & Machine Learning

- A Case Using Search in ChatGPT: Explores practical AI-powered search use cases in development workflows.
- DeepSeek V3: Multi-Head Latent Attention and Multi-Token Prediction: Technical deep dive into a modern LLM architecture.
- AI-Powered Git Commit Messages: Automating Git workflows with AI—useful for backend engineers managing large codebases.
- Using AI to Build SQL Queries: How AI can optimize database query generation.
- Testing Mistral API: Hands-on evaluation of Mistral's API for AI integrations.

Infrastructure & DevOps

- Automating Clash Proxy Management: Proxy automation for backend engineers managing distributed systems.
- Proxy Solutions in China - 2025: Updated proxy strategies relevant for engineers working in restricted regions.
- Managing AWS Lightsail Instances: Practical cloud instance management tips.
- Setting Up an Azure Server: Step-by-step guide for deploying backend services on Azure.
- Configuring OpenWRT as a Wireless Bridge: Networking optimizations for home/lab setups.

Backend Development & Tools

- Shared Objects in Multiple Threads: Concurrency challenges in backend systems.
- Java Bytecode: Low-level insights into JVM performance and bytecode manipulation.
- The Problems of Conda: Pitfalls of dependency management in Python environments.

- Git Large File Storage: Managing large assets in Git—critical for backend repos with binaries.

- Raw Http Requests And Wrappers: Best practices for HTTP clients in backend services.

Cloud & Networking

- Floating IPs in Hetzner Cloud: High-availability networking in cloud deployments.

- SSH Tunneling: Secure remote access techniques for backend engineers.

- Bandwidth Limit: Optimizing network performance in distributed systems.

- Hetzner Cloud: Review of a cost-effective cloud provider for backend workloads.

Automation & Scripting

- Automated News Summarization Bot: Building AI-driven automation for content processing.

- Mastering iOS Shortcuts: Cross-platform automation tricks (useful for DevOps).

- Useful Commands: Curated CLI snippets for backend tasks.

Performance & Optimization

- Apple Metal Performance: GPU acceleration insights (relevant for compute-heavy backends).

- Improving Wifi Speed: Network tuning for remote work/development.

- The Benefits of Repetition in Programming: How iterative coding improves system design.

These posts align with backend engineering interests, covering AI integration, infrastructure, cloud, and performance optimization. Let me know if you'd like more details on any topic!

Most Useful Websites and Apps for Me

Updated in August 2025

These are some links and apps I check daily or every few days. Typically, I use apps for updates, and for desktop use, I often rely on AI tools. They greatly impact my thinking. I like to find the best way to do things.

- ChatGPT
- OpenRouter
- Claude
- GitHub

- Apple
- Nvidia
- Google
- Grok
- Market Cap
- Binance
- Tiger Trade
- X
- Yin Wang's Blog
- Python
- Java
- WeChat
- TikTok
- JD.com
- WhatsApp
- YouTube
- My Blog
- USA.gov
- Yin Wang's Substack
- VS Code
- Ubuntu

On my home screen, I have apps for essential daily use, including the App Store, browser, news, to-do list, proxy app, AI chatbots, crypto trading, as well as banking and asset management apps. These are the tools I rely on for staying updated and organized.

Here are some of my most used apps on my main phone - an iOS device.

I can be selective about this, and few people will be offended. Most of these are famous or popular products, so they probably won't care whether I like them or not - I'm just one user among many. I also contribute to these platforms.

I include my personal blog website here. I check it frequently and update it sometimes. By doing so, I can better reflect on what I've done and share information I find useful with others.

img

Figure 1: img

img

Figure 2: img

img

Figure 3: img

img

Figure 4: img

LLM Costs, Agents, and Coding Tools

Table of Contents

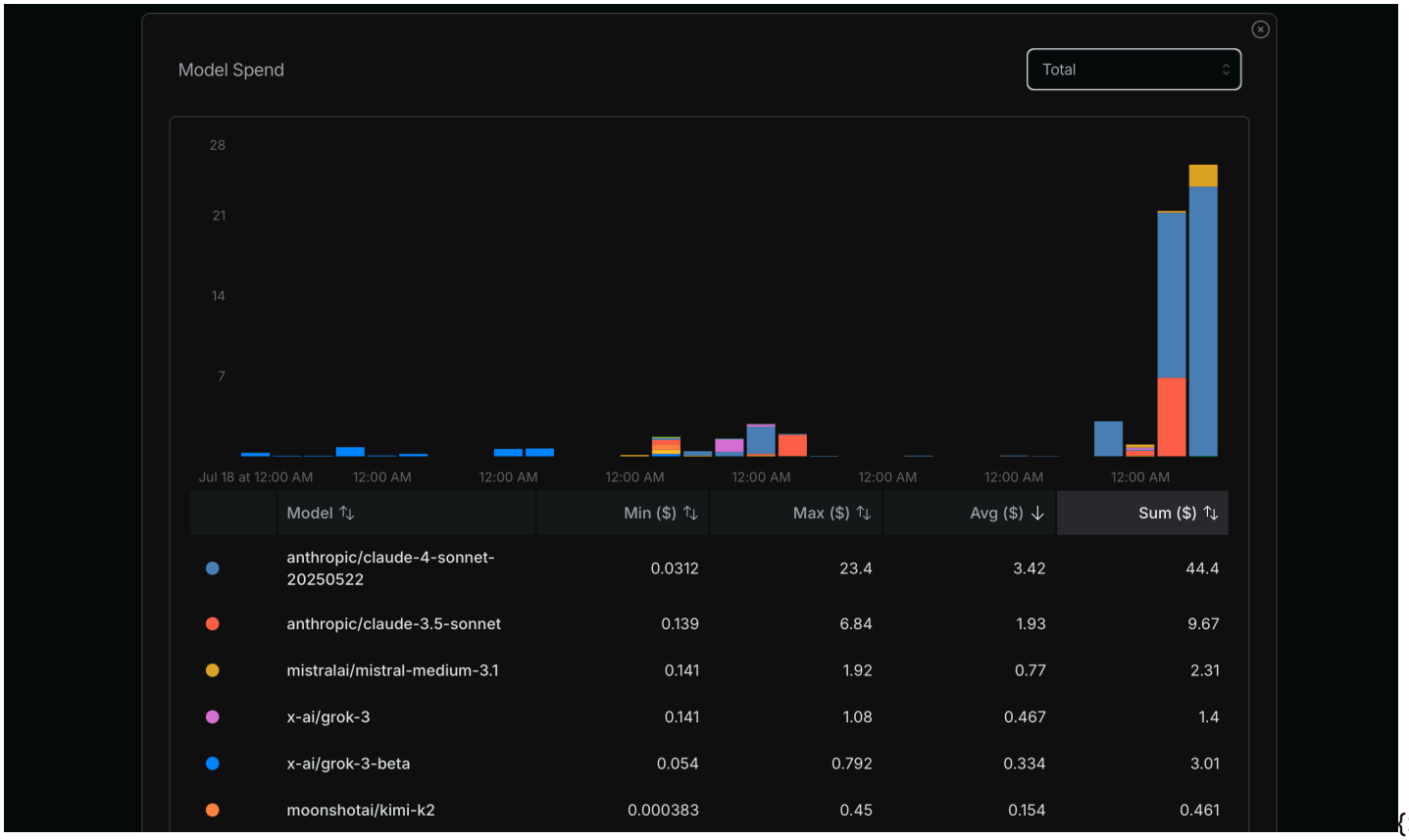
1. **Optimizing LLM API Costs**
    - Start with cost-effective models first
    - Avoid unnecessary high-end model usage
    - Prefer NLP libraries for simple tasks
    - Build specialized agents for efficiency
    - Compare models via extensive testing
2. **API Usage of Deepseek and Mistral**
    - DeepSeek costs scale with cache misses
    - Output tokens dominate Mistral expenses
    - Grok pricing heavily favors input tokens
    - Token usage varies by task complexity
    - Pricing aligns with documented rates
3. **General Agents vs Vertical Agents**
    - General agents struggle with complexity
    - Vertical agents excel in specialization
    - Workflow tools limit flexibility
    - Custom Python agents offer control
    - Trade-offs between convenience and power
4. **A Picky Engineer's Take on AI Coding Tools**
    - Prefer practical utility over brand hype
    - VSCode + Copilot remains reliable
    - Claude Code impresses with diff-style edits
    - Grammar tools require manual verification
    - Experimentation trumps blind adoption

Optimizing LLM API Costs

2025.08

{:

.responsive }



Model	Min ($)	Max ($)	Avg ($)	Sum ($)
anthropic/claude-4-sonnet-20250522	0.0312	23.4	3.42	44.4
anthropic/claude-3.5-sonnet	0.139	6.84	1.93	9.67
mistralai/mistral-medium-3.1	0.141	1.92	0.77	2.31
x-ai/grok-3	0.141	1.08	0.467	1.4
x-ai/grok-3-beta	0.054	0.792	0.334	3.01
moonshotai/kimi-k2	0.000383	0.45	0.154	0.461

{:

.responsive }

While optimizing token usage, it's advisable to start with more cost-effective models. Should issues arise, consider upgrading to more advanced models. Mistral, Gemini Flash, and DeepSeek are typically economical, whereas Claude Sonnet is generally more expensive. It's crucial to understand how Claude Code uses the routers shown below.

In my recent experience, I incurred significant costs due to neglecting this principle. I was trying to reach my maximum usage to determine the cost, which isn't a rational approach; it's a simple calculation. For instance, do I truly need Sonnet 4? Not necessarily. Although I perceive it as a more advanced model from Anthropic and it ranks highly on OpenRouter, I'm unclear about the differences between Sonnet 4 and Sonnet 3.5.

I learned something valuable from a recent interview with Replit founder, Amjad Masad. And for many tasks, highly ad-

vanced models aren't necessary. Ideally, if we can avoid using the LLM API altogether, that's perfect. Certain NLP libraries are effective for simpler tasks; for example, HanLP excels at handling Chinese language tasks.

Furthermore, we can develop custom or specialized agents to handle tasks efficiently from the outset. Claude Code might not always be the best or most cost-effective solution for every task.

One way to discern the differences between models is to use them extensively and compare their performance. After some time using Gemini 2.5 Flash, I find it to be less capable than Sonnet 4.

After a few days, I use the configuration below to help. The parameter longContextThreshold is really important. You can periodically clear the console of Claude Code, or restart it. It is very easy to hit the long context threshold when using Claude Code to write code.

```
{
  "PROXY_URL": "http://127.0.0.1:7890",
  "LOG": true,
  "Providers": [
    {
      "name": "openrouter",
      "api_base_url": "https://openrouter.ai/api/v1/chat/completions",
      "api_key": "",
      "models": [
        "moonshotai/kimi-k2",
        "anthropic/claude-sonnet-4",
        "anthropic/claude-3.5-sonnet",
        "anthropic/claude-3.7-sonnet:thinking",
        "anthropic/claude-opus-4",
        "google/gemini-2.5-flash",
        "google/gemini-2.5-pro",
        "deepseek/deepseek-chat-v3-0324",
        "deepseek/deepseek-chat-v3.1",
        "deepseek/deepseek-r1",
        "mistralai/mistral-medium-3.1",
        "qwen/qwen3-coder",
        "openai/gpt-oss-120b",
        "openai/gpt-5",
        "openai/gpt-5-mini",
        "x-ai/grok-3-mini",
        "x-ai/grok-4"
      ],
      "transformer": {
        "use": [
          "openrouter"
```

```
        ]
      }
    }
  ],
  "Router": {
    "default": "openrouter,x-ai/grok-4",
    "background": "openrouter,deepseek/deepseek-chat-v3.1",
    "think": "openrouter,qwen/qwen3-coder",
    "longContext": "openrouter,moonshotai/kimi-k2",
    "longContextThreshold": 2000,
    "webSearch": "openrouter,mistralai/mistral-medium-3.1"
  }
}
```

General Agents vs Vertical Agents

2025.08

Manus is claimed to be a general AI agent tool, but it probably won't work that well.

One reason is that it is very slow, doing a lot of unnecessary work and being inefficient. Another reason is that if it encounters a complex problem or hits a weak spot, you are likely to fail at your task.

Vertical agents work great because they are highly specialized. They are tailored for very specific tasks. There are dozens of databases and over a hundred web development frameworks like Spring. There are also numerous web frameworks, such as Vue or React.

Dify focuses on using AI to connect workflows, employing a drag-and-connect method to define AI workflows. They need to do a lot to connect information, data, and platforms.

I have built some simple agents too, such as a Python code refactoring agent, a grammar fixing agent, a bug fixing agent, and an essay merging agent.

Code is very flexible. So, Dify only covers a small portion of the space of possible ideas.

Manus performs tasks and shows users how it works by using a VNC method to display a computer.

I think the future will settle on these two approaches.

For Manus, you need to upload code or text to perform tasks, which is not convenient. With Dify, you need to build workflows using drag and drop, similar to MIT Scratch.

Why isn't Scratch as popular as Python? Because with Python, you can do so many things, while Scratch is limited to simple programs for educational purposes.

Dify probably has similar limitations.

Manus can handle a lot of simple tasks. However, for some tasks, especially those that hit Manus's weaknesses, it will fail.

Also, many programs or services take time to set up. In Manus's approach, this process is slow.

As a programmer, I use AI with Python to build my vertical agents. This is the simplest approach for me. I can also set up prompts and contexts to ensure relatively stable output from LLM APIs.

Manus and Dify are also built with these LLM APIs. Their advantage is that they already have a lot of tools or code ready to use.

If I want to build a Twitter bot agent, using Dify may be more convenient than building one myself with open-source technologies.

A Picky Engineer's Take on AI Coding Tools

2025.08

Recently, I successfully ran Claude Code, so I want to share my tool selection journey. I have also collected some AI Tool Tips along the way.

I was quite late to adopt Claude Code.

Claude Code was released around the end of Feb 2025.

I didn't succeed in trying it until recently. One reason is that it requires the Anthropic API, which doesn't support Chinese Visa cards.

Another reason is that Claude Code Router became available, which made my recent attempt successful.

I keep hearing praise for it. I tried the Gemini CLI in July 2025 but abandoned it after several failed attempts to get it to fix my code.

I also tried Aider, another software agent. I stopped using Cursor after about six months because many of its VSCode-based plugins malfunctioned. Additionally, I don't want to give Cursor much credit since it is built on top of VSCode. As the Copilot plugin in VSCode has recently improved and doesn't lag far behind, I prefer to use it more often.

However, VSCode is built on Electron, an open-source technology. It's challenging to attribute credit to the right team or individual. Considering that many large companies and startups profit from open-source projects, I must focus on my budget and what suits me best. I shouldn't worry too much about giving credit. I prefer using affordable and effective tools.

I briefly experimented with Cline but didn't adopt it.

I use the Copilot plugin in VSCode with a customized model, Grok 3 beta through OpenRouter, which works well.

I don't think Claude Code will change my habits, but since I can successfully run it and have the patience to try it a few more times, I'll see how I feel in the coming weeks.

I am a picky user with 10 years of software engineering experience. I hope tools can be great in actual use. I don't buy into the brand—I just care about daily usefulness.

After using Claude Code to fix this post's grammar, I've found it works well in certain scenarios. While I appreciate AI for grammar assistance (I even wrote a Python script to call LLM APIs for this purpose), I've noticed a frustrating pattern - even when I request minimal fixes, the tools keep surfacing numerous grammar suggestions for review. This manual verification process defeats the purpose of automation. As a compromise, I now let AI handle entire essays, though this approach limits my learning opportunities since I don't see the specific corrections being made.

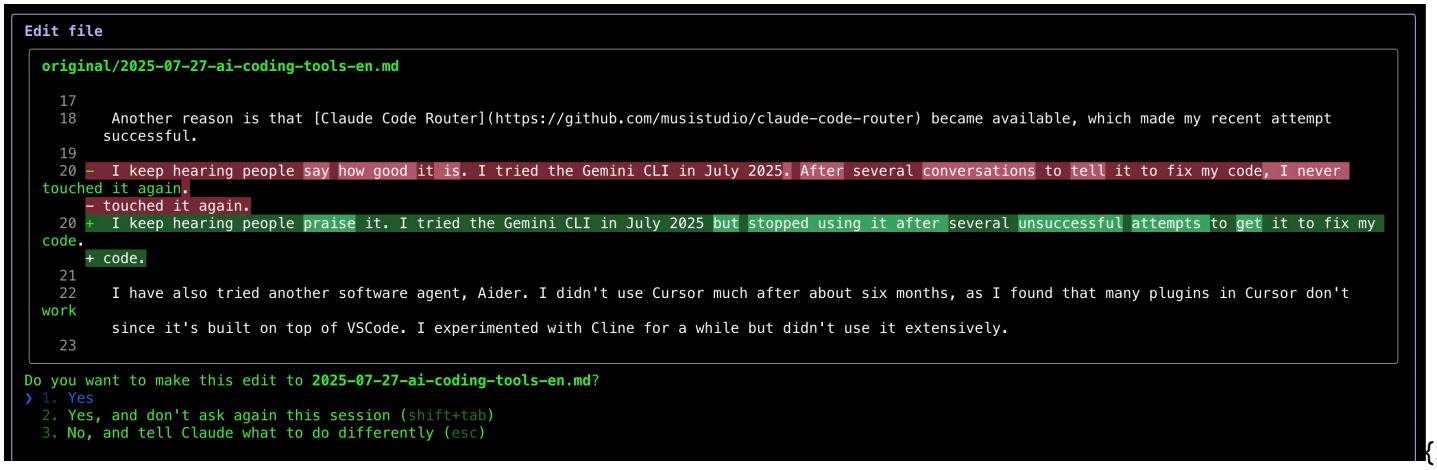
What impressed me most was how Claude Code displays changes - showing before-and-after comparisons similar to git diffs, which makes reviewing edits much easier.

After a day, I used Claude to fix some code as well. However, I continue to use the Copilot plugin with the Grok 3 beta model, as it is simple and easy for me.

After using Claude Code for several days, I have to say it's very impressive. I really like how it fixes my code.

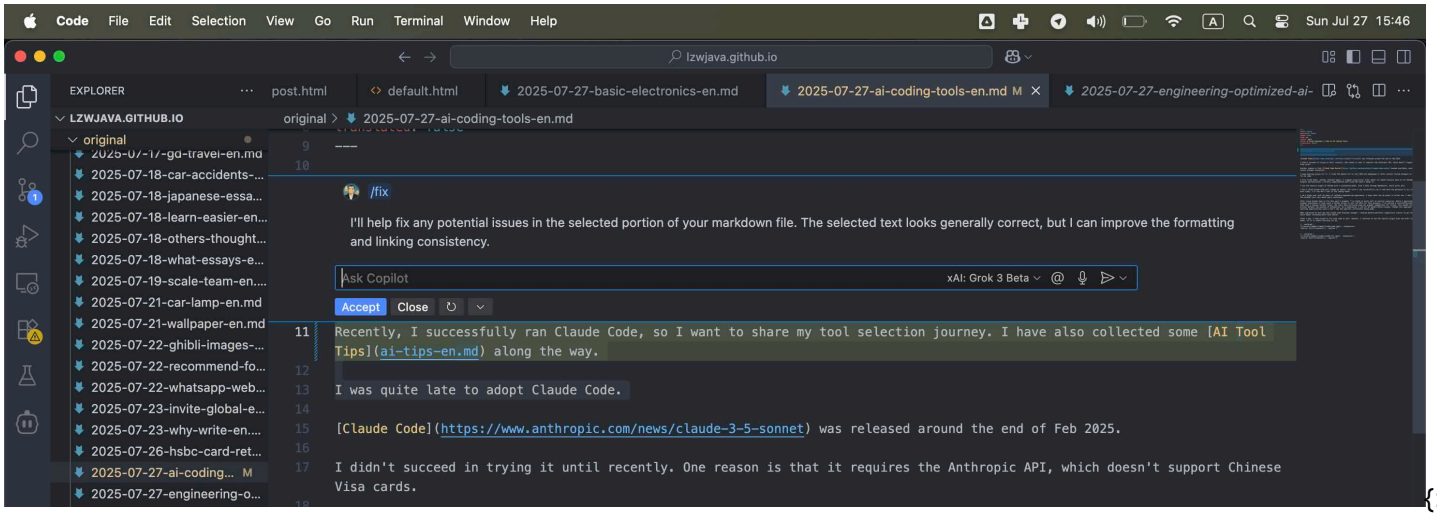
```
lzwjava@Zhiweis-MacBook-Air-1255 lzwjava.github.io % ccr code

 * Welcome to Claude Code!

   /help for help, /status for your current setup

   cwd: /Users/lzwjava/projects/lzwjava.github.io

   ────────────────────────────────────────────────

   Overrides (via env):

   • API timeout: 600000ms
   • API Base URL: http://127.0.0.1:3456

 ※ Tip: Hit Enter to queue up additional messages while Claude is working.

> hi

● Hi

> what is the readme file ? could you summary

● Read(README.md)
  └ Read 121 lines (ctrl+r to expand)

● Update Todos
  └ ☐ Summarize the README.md file

● Update Todos
  └ ☐ Summarize the README.md file

● The README.md file describes a personal blog built with Jekyll, featuring AI-powered translation,
  MathJax support, night mode, and more. It includes setup instructions, markdown styling tips, and

 >
```

{: .responsive }



{:

.responsive }



{:

.responsive }

Api Usage Of Deepseek And Mistral

2025.01.25

DeepSeek

In one month, 15 million tokens cost me approximately 23.5 CNY.

This was my usage in one day:

Type	Tokens
Input (Cache Hit)	946,816
Input (Cache Miss)	2,753,752
Output	3,100,977

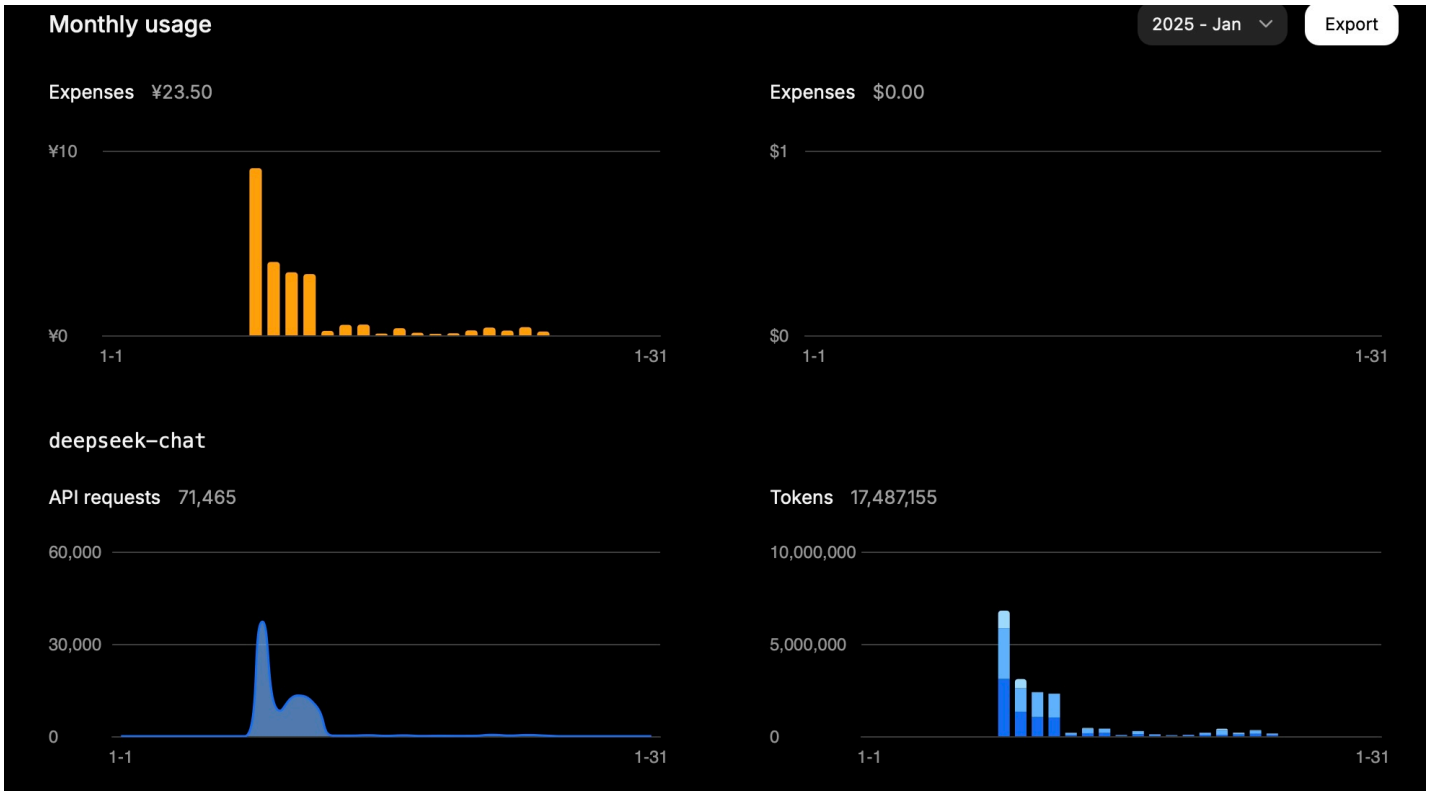
The calculation is as follows:

0.94 * 0.1 + 2.75 * 1 + 3.10 * 2 = 11.83

So, depending on the task, the token usage largely depends on the input (cache miss) and output.

This result aligns with the expected cost.

DeepSeek API Pricing



Mistral

The pricing for Mistral models is as follows:

Model	Input (USD per million tokens)	Output (USD per million tokens)
`mistral-large-2411`	2	6
`mistral-small-latest`	0.2	0.6

In one day, my Mistral account usage was as follows (Model: `mistral-large-2411`):

Type	Tokens	Cost (USD)
Total	772,284	3.44
Output	474,855	2.85
Input	297,429	0.59

For the `mistral-small-2409` model, the total usage was 1,022,407 tokens.

Assuming 1/3 of these were input tokens and 2/3 were output tokens:

There were 340,802 input tokens and 681,605 output tokens.

Therefore, the total cost is calculated as 340,802 * 0.2 / 1,000,000 + 681,605 * 0.6 / 1,000,000 = 0.07 + 0.41 = 0.48 USD.

The Mistral console reports a total usage cost of 0.43 USD, which approximately matches our calculation.

Grok

Model	Input (USD per million tokens)	Output (USD per million tokens)
`grok-2-latest`	2	10

Optimizing Software Engineering for AI

Table of Contents

1. Optimizing Software Engineering for AI
    - Flat Architecture for AI-First Development
    - Python's Advantages in AI-Driven Workflows
    - Context Engineering and Token Optimization
    - Structuring Code for AI Assistance
2. Thriving as a Manual AI Agent
    - Working with AI Tools in Enterprise Environments
    - Tool Selection and Context Management
    - Building Reusable Prompt Systems
3. Leveraging Python for Java Development
    - Python Scripts for Java Project Support
    - Cross-Language Development Strategies
    - AI-Assisted Code Generation
4. Programming Languages in the AI Era
    - Future of Python, Rust, and Java
    - Performance vs. Simplicity Trade-offs
    - Language Evolution and AI Integration

Optimizing Software Engineering for AI

In this blog, I utilized hundreds of scripts to assist with translation, playground, frontmatter maintenance, and Telegram bots. I believe this development approach might represent the future of software engineering optimized for AI.

I don't rely much on Python module functionality, nor do I want to structure code like a large Java Spring project.

I have worked on many software projects throughout my career. I have observed impressive banking architectures, microservices, effective multi-country designs that minimize duplication, robust foundational frameworks built on top of Spring, and strong governance with centralized configuration.

While these banking architectures are impressive, if we were to start today, I would consider optimizing for LLMs and AI. This would involve better context engineering, improved separation of concerns, and prioritizing AI-first thinking over human-centric design. Although Spring offers multiple layers and good abstraction, it can be challenging for LLMs and AI to navigate.

I believe we should aim for flatter structures, similar to a flat organization. This means using just two levels: the first level calls the second level. In a function, it's better to call another 50 functions directly rather than having 50 nested levels or stacks. AI/LLM struggle to judge or infer overly complex, nested structures, but they excel at handling smaller functions of 100 to 200 lines of code. Python is well-suited for calling and importing from other files.

One reason Python code is easier than Java is that its dependency management is simple. You just need to use `pip install` to add a dependency. With Maven, you need to write the dependency in a POM XML file and then use `mvn compile` to let Maven download the dependencies.

Another reason for Python's simplicity is that its code can be directly run without hassle.

Though starting with Java 11, the `java` command can directly execute single-file source-code programs without needing to compile them separately using `javac`. However, often for Java projects, they are large, so one has to run them with `mvn spring-boot:run` along with some properties configuration.

A third reason is that Python's module design is simple; you can use `from` and `import` to easily import code from other files.

For now, many AI chatbots can run Python code directly in the chatbot window, like Grok.

When comparing 100 Java files, each with around 1000 lines of code, to some simple Python scripts, it is not a fair comparison. For this kind of project, I would prefer to have 1000 Python files, each with around 100 lines of code.

It is acceptable to select lines of code or a function to edit. However, you need to know where to select. Why not let this task be handled by AI to make our lives easier? So, we just need to use "select all"to select all the code and tell AI/LLM how to edit.

For Python, it is easier to use `if __name__ == "__main__":` to run and test functions in a file. It is also easier for other Python files to call the functions inside this file without needing to run the test.

This is context engineering optimized for AI. Could we approach it in other ways? AI/LLM is auto-regressive. However, when we use Copilot or Claude Code, we don't know how the AI Software Agent helps us. They should think about it instead of us.

Could we, on the user side, arrange code specifically to reduce token usage? For this point, the approach of having 1000 Python files with each 100 lines of code is good for this purpose. Because you can verify functions and code files easily before letting other Python code call them.

But one problem is that if you want to change several files of code together, it is not easy to do so. For a simple way, you can copy the code to AI chatbots and let them tell you how to edit code in these files.

Possibly, we don't need to use the number of lines to separate functions or logic. But we should do it to separate logic into small functions. We can do this by naturally separating them by type of logic, so they appear shorter.

Why do we want Software Engineering Optimized for AI? Because AI is powerful, we should optimize everything for AI and then let AI help software engineering as much as possible.

It is possible not just for code, but for any text too. Suppose we are very picky editors; we don't want AI to edit our large texts at once. We want to check paragraph by paragraph. For code, we can tolerate minor errors or bugs. For text, we can tolerate them because most readers are not so picky.

But code is different in that sometimes, even a minor error can lead to the complete malfunction of a large project.

For XML or YAML files, we probably don't need to separate them as much because they are already highly structured.

And for HTML files, we should do some separation. Instead of writing hundreds of JavaScript files together with hundreds of HTML files, making it easy to exceed 1000 lines of code, we should use `import` for JavaScript to manage this. For JavaScript code, we can use the above ways to separate.

We want to structure code in ways that let AI help us easily add, edit, delete, and run code. This is the beginning. Think of a day when all code can be generated or fixed by AI easily. The world will be highly digitalized.

Imagine myself writing 100 big software projects and providing APIs to connect with others. This includes my daily agenda; I am, myself, like a 1000-employee tech company nowadays. They are customized for my needs, to earn money or spend money for my benefits. This is really amazing.

Thriving as a Manual AI Agent

AI Agents should be automatically run with code. Now, the title of this essay is "Manual AI Agent."You might think I am kidding, but I am not.

The reason I say "manual AI agent"is that for big companies, technology adoption is slow due to security data concerns and long-term considerations.

There are a lot of new technologies in the market; who knows what will last or what will disappear quickly.

They also have security data concerns. Typically, they want to partner with big brands whose data policies are strict and monitored by the public. This explains why Microsoft has become a top partner among Fortune 500 companies. Other companies use their Teams, Microsoft Office 365, Azure, and Copilot.

However, what if big companies do not provide their employees with LLM APIs to use? We need to think about how to work as manual AI agents.

That means we will use a lot of tools to work, similar to tool use or function calling in those APIs. We will do our own prompt engineering or context engineering.

Instead of using Claude Code or Manus to do a complex task, we may perform tasks ourselves with a plain AI chatbot.

AspectJ is good because it uses AOP programming to intercept methods. Filters in Spring are also good for capturing the logs of HTTP requests. The logger in Log4j is good for redirecting specific logs to a file. IntelliJ IDEA is good because it has a function to export objects as text.

SQL clients are good because they can easily export CSV or Excel files of rows. Git diff is good because it can give you comparison text.

They all help you provide better context for AI chatbots. And AI chatbots can also help a lot of Python scripts to perform tasks.

To be an effective AI agent, you need to use many effective tools to help you do tasks, whether simple or complex.

Without APIs for LLM/AI chatbots, you need to copy text into the chatbots. It is a little more tedious than directly calling the AI, but the good news is that you can select context or prompts more carefully.

So you do not need to ask AI chatbots many times like those automatic AI agents. You can carefully select the tools that you will use.

So working like a manual AI agent has its benefits. However, AI agent technology evolves fast and shows its potential to the world.

If they are very useful, big companies will adopt them just like AI chatbots. Otherwise, they cannot compete with other companies that have adopted them—not just other big companies, but also small startups. Because AI is so powerful now, a startup with tens of employees may beat those with 1,000 employees.

Working as manual AI agents is sometimes unavoidable. The job has other benefits besides lacking advanced AI technology. It is not easy to find good jobs, too. So in this case, it happens to give us space to use our traditional wisdom to make the most use of AI chatbots.

And it means that we can organize and accumulate our prompts to create system prompts for AI chatbots, similar to those used by Claude or Grok that have been exposed. This way, we don't need to write prompts repeatedly. We can use Python scripts to assist us in writing prompts. We can obtain the logs of HTTP requests and write prompts to generate API test cases.

The magic of programming lies in its unlimited levels of abstraction. It's akin to functions where you can have 100 levels of function calls. For instance, WeChat is built on top of iOS, and WeChat Mini Programs are built on top of WeChat. iOS itself is built on top of Objective-C or Swift, which in turn are built on top of LLVM and the instruction set of Apple's ARM chips.

Leveraging Python for Java Development

How to use Python to help Java development in the AI era? I like Python. I have worked with Python the most in the last around 3 years, since ChatGPT was released at the end of November 2022.

One way to help is using Python to write SQL helper scripts, test scripts, and log searching scripts for Java projects.

Use Python to analyze POM files and package dependencies for Java. Use Python to check data consistency in Java. There are a lot of things we can do in Python instead of Java.

But Java doesn't have PyTorch. Python can help with anything in 200 lines of code that would take 500 lines in Java. But by using AI tools, you can't easily get your own version of PyTorch either. Even something like TinyGrad takes time to build.

Why write our own scripts first? One reason is that it is super customizable. There are no public software or open-source projects that can help us directly in our projects, especially those in big companies.

The big projects in big companies are developed over one decade or more. They already have a lot of customization.

So in the future, there will be a lot of surrounding projects around the big projects in big companies. There will be more Claude-like code routers in internal coding agent tools in big companies. There will be more customized Postman, SQL clients, and compilers for big companies.

Using Python code can connect to Java agents too.

It means that I need to learn Python and Java well, so that I know how to use one to help the other.

And I can use Python with the help of AI to create a lot of things for myself and in corporate projects too. Java seems like no obstacle. Java, with Spring, databases, and with Angular, Vue, or React serving as frontend, shouldn't be an obstacle to having Python help a lot.

Programming is such a flexible thing. The limit is our imagination.

So AI is growing fast. We can measure AI's progress by how much and how easily we can use code to achieve things with AI's help in coding and learning.

Could we one day write some AI agents, and then these agents help create an entire TikTok, including its lots of microservices and big iOS or Android projects?

If AI is so powerful, what should we do today? Probably nothing, as what we do today will be easy to implement with AI. In 2025, our 1 year's work with the help of AI probably can be done in 1 month with 2030's AI ability.

This brings up our essential question: What's our life purpose? What's all this about? How to live a good life?

AI is coming out like other technologies to bring us freedom. But it seems everyone is busy like a machine in this capital society.

Coming back to the topic. So Python can help write Java code too. You can use Python to get the context for writing code and let Copilot write it for you to do it right on the first shot.

AI is about prompt engineering and context engineering. Prompts and context help the responses of AI chatbots.

Python can help with context; Python can help generate prompts.

So this is not just about Java, but every other programming language. Python can help them deeply. So why do we still need to use other programming languages?

It is Python's intrinsic design that makes it perform poorer than other programming languages, like C, C++, or Rust.

Programming Languages in the AI Era

AI is so powerful now that we have to rethink everything from the perspective of AI. Which programming languages will be popular in the next 10 years?

Python surely will be. Many AI chatbots use Python to execute code in the browser, like Grok. Python is popular for its simplicity, ease of learning, and decent performance. It is adopted by many software projects.

Python is slower than C++, Java, and Rust. Java has a large community. Rust is built on top of C.

I wonder whether many projects will be rewritten or replaced by Rust. Being rewritten in Rust means referring to an old project and using Rust to implement the same functionality. Being replaced means that software written in other languages is now replaced by similar software written in Rust.

Rust has a relatively complex syntax. But in the AI era, that is not a big problem, because AI will help write code. For complex syntax, humans don't actually have much trouble either.

I think Hindi or Tamil are quite complex. But for Indians who live in the North, Hindi is not a problem, and for those in the South, Tamil is not a problem either.

But for a Chinese citizen like me, I think it is a big problem to learn.

At first glance, all characters in Hindi look similar to me. I think the difference between Hindi and Arabic is like the difference between Chinese and Japanese, or English and Spanish.

The differences between programming languages are less than those between natural languages. One big reason is that programming languages only differ in character appearance, while natural languages also differ in sound. Natural languages differ in two aspects: character set and pronunciation.

Programming languages have only about a century of history, but natural languages have more than 100 centuries. The more time people spend on something, the more differences develop. People with slightly different opinions will create their own versions of things.

That explains the English accent. In some TikTok videos, people say the worst English accent is Birmingham.

So actually, Rust doesn't have much of a problem. Its performance is quite good, as it is built on top of C/C++.

Performance is critical for many applications. Nowadays, many apps are used by billions of people. For underlying cloud computing infrastructure, their services are called many times. So even a small performance gain can save a lot of money.

Does Rust have many disadvantages? One thing people complain about is that it is hard to learn. The learning curve is steep. AI brings good news, as it helps with learning a lot.

I don't need to know much about Rust. As a software engineer with 10 years of experience, I can use AI to help write many simple Rust applications. I just need to know the basic Rust compilation commands like `cargo` and `cargo build`. I don't even need to know much about Rust syntax itself.

For Rust, the mutability or borrow model doesn't cause trouble for me. For simple applications with fewer than 200 lines of code, I can ask AI to fix errors directly by providing error messages.

But why do people still use Python a lot if Rust is that good? Because Python is good in another aspect. It is very easy to use and learn. It has a large community and many libraries.

Python still has good enough performance and can support products for millions, even tens of millions, of users. Most products don't get that many users. If you do have that many users, you can hire Rust or Java programmers to optimize performance.

Python is good for a lot of development: machine learning, web development, math, teaching, and scripting. While Python is not good at desktop applications, MicroPython is used in Raspberry Pi.

How about Java in the AI era? It will be good too, as it has a large user base and community. AI helps a lot with that. It is used by many big companies. They tend not to change their major programming languages. For some of their large legacy projects, using a new programming language to rewrite a project would take a decade of effort. AI will help with that, but the process will still be slow.

Often, rational people in big companies won't consider changing their main programming language. Their major business is in other sectors. They don't care much about technology. If they did, they would become software or Internet companies and lead in open source communities. However, not many Fortune 500 companies care about that.

There will be a lot of startups due to AI. Startups like to do new things, so they will try new programming languages. In the AI era, agile programming languages will win in small and medium enterprises.

In algorithm competitions, will the favorite programming language change? C++ has dominated this sector for decades. In actual algorithm competitions, you can't use AI. But I think that in the AI era, fewer people will participate.

Since those people are very good at programming, and there are so many opportunities due to AI, why wouldn't more people build actual products for users instead of practicing algorithm problems? Even the GOAT of algorithm competitions, Gennady Korotkevich, chose to join Devin.

But algorithm competitions can be a relaxing or retiring hobby for smart programmers. It is just like chess or basketball. People do it because they like it or need it, not for other reasons. Many people play basketball in their 30s or 40s. They probably do it for health reasons or to make life more fun.

For iOS and Android, it is Java, Kotlin, Swift, and Objective-C. There won't be significant changes due to AI, because there are limited choices. On the end-user side, performance requirements are not so high. Google and Apple have very high control over their platforms. If Google and Apple don't change, programmers won't change.

But for servers, there are many choices. Languages that are more AI-friendly will win.

Procedural programming languages will win more than object-oriented languages. Procedural languages are direct and easy to generate by AI, while OOP languages have many nested levels or design patterns.

Will there be more programming languages because of AI? I think so. Zed and OCaml will have more users. LLMs/AI are very good at learning patterns, so it is easy to rewrite projects in other languages.

Programming languages will face more competition in the future. Those that are good at performance, syntax, and compiler quality will inherently become more popular. The competition is just like with LLMs. Those that are inherently good, like Claude and DeepSeek, become popular.

What if AI becomes so powerful that we don't need to learn programming anymore? That is still a long way off. Suppose we have a very big project with 1,000 Java files. AI would probably need 10 years to perform tasks on that easily.

Complete Guide to Learning English

Table of Contents

1. Affordable English Practice with Native Speakers
    - Costs 18 CNY per 25-minute session

    - Focuses on IELTS speaking exercises
    - Uses Pinduoduo for course comparisons
    - Teachers are Filipino natives
    - Goal: improve from IELTS 6 to 7.5
2. Mastering English
    - Dedicated 16 hours daily learning
    - Transformed brain through immersion
    - Used English in daily communication
    - Overcame dropout limitations via skills
    - Combined learning with daily activities
3. The Recipe to Learn English
    - Learned via Google, TikTok, Netflix
    - Focused on enjoyable immersion methods
    - Improved typing speed from 5 to 20 wpm
    - Avoided painful exam preparation
    - Applied method to other subjects
4. Improving English with Extensive Input
    - Shifted from Chinese to English content
    - Used songs, quotes, games for learning
    - Watched 100+ shows without subtitles
    - Spent 8 hours daily on English
    - Emphasized natural, painless learning
5. The Journey of Learning English
    - Started with Paul Graham's essays
    - Used TikTok as primary learning tool
    - Read 15+ English books patiently
    - Practiced via Upwork client projects
    - Focused on immersion over memorization

Affordable English Practice with Native Speakers

2025.02.15

Recently, I tried an English language conversation practice session with a native speaker from Pinduoduo.com.

The assistant will add your WeChat via your mobile phone number and help you arrange the course.

It's cheap, at only 18 CNY per course for about 25 minutes. The teachers are from the Phillipines. By the way, I just learned the word "Phillipinos"from this chat.

I discovered this course through a post on X and was drawn to its affordability. Despite achieving an IELTS score of 6 two years prior, my goal is to reach 7.5 this year, necessitating increased English practice, particularly in speaking and writing.

We talked about some IELTS speaking exercises. The English teacher asked me exam questions about topics like travel,

photo-taking habits, and traffic jams.

Another famous platform, VIPKID, charged 150 CNY per course for 25 minutes years ago.

So, the price is really cheap here now.

I searched for Japanese conversation practice on Pinduoduo.com. Unfortunately, there are no merchants offering such a service at the moment.

I would like to take more English courses.

{: .responsive }

I bought 10 courses for 180 CNY and tried another merchant on Pinduoduo.com. I prefer to buy and compare products or

services on Pinduoduo.

Mastering English

2022.11.08

One year ago, regarding my English ability, my reading was good, my listening was not so good, and my writing and speaking were poor. After working in a Singaporean company, four aspects of my English level are proficiency.

I passed a lot of interview rounds recently, and got several offers from big outsourcing companies whose clients are big banks. Though I will work still as a contractor for a big bank, I am still proud of what I achieved regarding that I was a dropout student.

Regarding the poor economic environment in China, I am confident that I will not lose my job in the next 10 years. Many graduated guys lose their jobs. It is hard for many middle-aged people to find a job. Society is ruthless. I am also trying very hard to ensure that I can find a job and pay my mortgage.

I learned English a hard way. Besides my 965 work, all of my leisure hours are spent learning English. 16 hours on the weekend, 8 hours on the work days. After half a year, I find my brain changed miraculously. I become passionate about speaking and writing English, become passionate about listening and reading English. I must ensure that I have some English materials to listen to whenever my mind is awake. Of course, it is annoying when you listen to lectures and write some code at the same time.

I love expressing myself in English now. As long as my Chinese friends can read simple English, I will send English text messages to them. I made bilingual short videos, so I made bilingual captions and dubbing. I just changed myself to an English native speaker though I live in China surrounding the Chinese.

Why? One reason is that as now I am proficient in English, it is easy for me to speak and write. Another reason is that I want to make my English level perfect. And I want to inspire more of my friends to reach the English level I got. The last reason is that using a lot of what you learned is funny. Especially not many people can do that, it is cool. And just one year ago, I couldn't do that. So, see, how miraculous it is.

So how can I learn English for 16 hours a day? If I can play mobile phone, I will happily play TikTok and Youtube Shorts. The short videos are funny. If I can't play mobile phone, I will listen to podcasts on Google Podcasts and listen to Youtube videos which you can play in the background if you are a Youtube premium member.

And what am I doing when I am listening to English lectures at the same time? Listen when driving, listen when laying in bed, listen when falling asleep, listen when eating meals, listen when walking, listen when riding a bicycle, listen when talking to others, listen when going to the toilet. So just let the English lectures play all the time, and focus a little bit some time to listen carefully and try to understand. I find in this way, I learn quite fast.

So now, my daily English and working English are proficient. I am about to make my academic English proficient. How can I do that? Same way, try to listen to a lot of academic materials. I am confident that my academic English can be quite sharp after one year.

In the above paragraph, I mention I listen to English lectures when talking to others. How can I do that? So it is on-ear headphones.

So you can hear what others say clearly at the same time you can hear the voice from the headphone. And what other tools do I use?

I find that if I wear Bluetooth earphones too much, I will feel pain. It may be that I listen for too much time. So I bought a mobile phone rope to wear on my neck. So I can use my mobile phone to listen to lectures when there are not many people surrounding me. I will preferably use this way more than the on-ear headphone way.

I try hard to solve the problem of how can I live well as a dropout undergraduate student. Now I can easily find a job. The global big company cares more about the English level of candidates instead of their college diplomas. And there is a lot of outsourcing company whose clients are global big companies.

And actually, I am just one step to a permanent position at a big bank. I failed in the last round of the interview. They weren't unsatisfied with my English level. They mentioned that my technical knowledge foundation is quite good, however, I still lack a bit of knowledge deepness.

So my answer is that if you can do the things the people who have a college diploma can't do, then you can beat them. Especially in the highly competitive and hardcore technical area, there are many chances for dropout, for the people who want to prove themselves, and for the people who want to earn a high salary.

So I am trying hard to solve my dropout problem. I spent half a year learning 16 hours a day to solve them. However, the great lesson is that if I can learn English so fast and well, why can't I learn other things as well?

So I am trying to polish my academic English comprehension ability. I am listening to academic lectures the whole day. I listen to lectures about how to make a rocket, how the economy works by Ray Dalio, and how to make a computer from scratch. I listen to one lecture for a whole week. At the first shot, I don't know what they are talking about. After a week, to my surprise, I can almost listen most of the words they speak and understand some main points they share. It is quite amazing. Because I originally don't think I can learn such stuff, especially since they are taught in English.

So how to become the sharpest person in the world? I think I find the answer. Just learn the whole day, the whole week, the whole year. You will be quite sharp. I will be quite sharp.

The most difficult obstacle for me to becoming the sharpest is that I am an easily distracted guy. My concentration ability is poor. However, the good news is that when I play lectures for 2 hours, to let the voice surround my ear, then I can focus a little bit to listen carefully for 1 hour. Especially when you are driving, your eyes are looking ahead, and you are bored, so your mind has just enough attention to listen and learn something. When I drive for 2 hours, I can listen carefully for 1.5 hours.

Is it amazing? Yes, it is quite amazing. And nowadays, new energy vehicles are becoming popular. I also bought one, NETA V Chao Lite 400. I think it is the best new energy car whose price is below 100k CNY. Young people like me in China can afford one. The car is amazing. So you drive to do a road trip without spending much money to get fuel for the car.

And in this way, I can only get familiar with technical stuff or academic papers. Regarding most of the precious knowledge, I must do to learn. How can I learn precious knowledge?

I find a way today. When I did something, I got anxious when I feel I am not progressing. For example, when I build some

circuits or play with Arduino, I find my progress is slow. Then I give up. Then, I find my solution. I listened to English lectures while building circuits, in this way, I get to brush up on my English even if the circuit doesn't work. At least I get one percent more familiar with some words and some sentences in English. So I think if I play lectures at my home using a Bluetooth speaker, then I can build circuits the whole day. If I want to have the patience to do some labor work, then I just need to play lectures to make myself relaxed to do it.

Does it sound exciting? Yes. It is really exciting to learn if you can learn very fast and without much effort. It is appealing if you have ways to learn anything. It is wonderful if you can calm down to do any labor experiment or work.

This is my way how to becoming the sharpest person in the world. Because I learn 16 hours a day, I experiment 16 hours a day, I explore 16 hours a day.

I have a daughter who is one and a half years old now. Her father learned for 27 years and used 2 hours to write the most precious lesson down. So it is dedicated to her. I am 27 years old now. It is hard for me to become the sharpest though it is still not late and I have chances. But I am very confident that my daughter will become one of the sharpest people when she grows up.

One generation beats one generation. Humans are wonderful.

The Recipe to Learn English

2022.09.30

Recently, I shared my job change with my friends. To their surprise, the big change in me maybe my English skill improved a lot in a year.

I already share the article about how to learn English. Let me summarize my total effort to learn English after graduating from high school.

7 years to use Google search to read English websites. Half a year to play in TikTok or Youtube Shorts for at least 6 hours. Watch Netflix for around 100 TV shows and movies. Read about 30 English books. Use English to work in an international big company. Listen to Google Podcast for at least 4 hours in half a year.

See, these are my total effort. Does it seem that it is really hard work? It may seem like a lot of work. However, I don't feel any pain in this process. Do you learn your mother language painfully? No, you don't. So I just make learning English like learning my mother language.

I think everyone in Chinese can reach the level I got. It helps me to pass the English technical interview at a big company. I heard that only in 1 of 50 developers, there is just one guy who can speak English fluently.

I joined a big Singaporean bank as a contractor. Before I joined the company, my reading is good, my listening is not so good, and my writing and speaking is poor.

I know that when at the first day of my work, I only type English words for 5 words in a minute. However, after several months, I found that I can type 20 words in a minute. I still have some grammar errors in my writing. However, at least, I can express what I want to say and my colleagues can understand well.

My colleagues say that the English environment is really important. Yes, it is. However, these days, we can use the Internet to mock the English environment. Youtube, Netflix, and Google Podcast is really good.

So I took a TOFEL Exam. However, in the process of taking the exam, my computer adapter isn't plugged in and the screen is turned off. After I plug in the adapter for 15 seconds, the teacher can't see me. So I canceled the test. I will retest the exam in the golden week. In a mock exam, I can get 80 to 90 scores of 120. However, I am afraid that in the real exam I can get that points. Listening and reading are hard. Also, I don't like to prepare for the exam. The idea that we sit before the computer and practice for the exam as a student will kill me. It is so painful. I don't have that kind of self-discipline.

I can use my way to learn English and I feel that my English skill improve a lot. So why do I try to prepare for the exam to do better in the exam? I know there are a lot of reasons. But I don't like that way.

So when I feel that my English ability improve a lot. However, the test beat me. So I begin to listen to podcasts during my daily commute. So I try to walk for 2 or 3 hours daily to wear Bluetooth earphones to listen to podcasts. The Lex Friedman Podcast is really good. I listen to the speeches of Steve Jobs and Tony Fadell a lot.

So I just listen to podcasts for around 5 hours a day. After several months, I notice my brain had changed. I can use English to express myself faster. When I want to say something, my brain quickly knows how to say it. Just like using Chinese.

So then I think if I can use this way to learn English, why not Japanese or Hardware? or whatever you like.

My insight is that your brain is easy to distract. So if you let your brain in an environment, then it can focus sometimes. So if I let my brain emerge in 5 hours of English talk, then my brain can focus to listen to it for about 2 hours. Then I can feel that I learned fast!

So this is my recipe to work hard. And I start to learn hardware and chips. I listen to the capacitor and transistor the whole day. So I can think about it again and again. The electricity is just like a flow, the electrical component is just like the software library. The electrical parameter is just like the API of the software. You can gain more insights after thinking and listening to them for long hours during the day.

I am a very lazy person. If I can do it, anyone can do it. In the modern highly competitive day, you need to work hard to be great. I am glad that I find a way to work hard. And it is very funny. I don't feel pain.

See, I just use one hour to write this article. And just half a year ago, I feel very painful to write one sentence myself.

Let's end with my recent favorite quote.

> The difference between a good student and a great one is that a good student is concerned more about the outcome while a great one is fascinated by the process of learning.

And, friends, long time no see, hope you guys all doing well.

Improving English with Extensive Input

2021.01.25. This post was originally written in Chinese. It has been translated to English to facilitate further translations into other languages.

In 2014, when I encountered programming problems, I would search for English web pages on Google. However, I found it difficult to continue because I read slowly and frequently needed to refer to a dictionary. I felt lost in the clouds and decided to turn to Chinese web pages instead. At that time, I preferred reading in Chinese whenever possible. However, over the past six years, my approach has reversed, and I now only read content in Chinese when it is not available in English. I spend my leisure time in the English-speaking world, watching American TV shows without subtitles and browsing TikTok videos. Despite my efforts, my English proficiency is still not as good as my Chinese, maybe only half as good.

Regarding the question of "How to learn English to the level of a native speaker"I believe I have found the answer. Over the last two years, my English proficiency has steadily improved, especially during the second half of 2019. Looking back, I realize that if I had put in the same effort in 2015-2016, my English proficiency might be as good as it is today. With time and work, I can now use English resources to solve many of the problems I had given up on before. This progress allows me to read more professional books and broadens my horizons. In the internet age, I have come to realize that we can simulate living in English-speaking countries without leaving our homes. People in English-speaking countries are similar to us; they spend most of their time on phones and computers, surfing the internet. Once we learn how to access the internet freely, we can enter their world too.

I am sharing these insights and experiences as if I am speaking to my younger self, to high school and college students today, and to anyone interested in learning English on their own. In my previous article on "The Journey of Learning English,"I discussed my experiences in learning English over the years. Here, I want to add some recent reflections.

Just like with any other subject, the most important thing is finding a method that you enjoy and sticking with it. If you love English songs, start by learning their lyrics, word by word, understanding their meanings. Then, listen to the songs and write down the lyrics with a pen or on your computer. If that seems challenging, that's okay; move on to learning another song. Learn many songs, maybe a hundred of your favorites. Believe me, when you have learned a hundred songs, your brain will be different. You'll find that you can understand similar lyrics and songs much easier. To continue improving your English to a native-like level, learn another hundred songs, but this time, switch to different topics and genres, from love songs to country music.

If you enjoy quotes, find a hundred inspiring quotes, especially from people you admire. Learn them one by one, look up words in the dictionary, and understand their meanings. If you admire certain foreign personalities, follow them on Twitter, read their tweets one by one, and look up unfamiliar words. If you like playing games and also want to learn English, try playing English-language games. Find a hundred games to play. If you love watching American TV shows, try watching them without subtitles. Initially, it might be challenging to understand, but as you continue watching, you'll gradually notice that you can understand more. If watching without subtitles is too difficult, try using English subtitles. Platforms like Netflix and YouTube offer subtitle options easily. After watching a hundred episodes of TV shows or documentaries, you'll realize that your English listening skills have improved significantly. If you continue this for a couple of months, you'll find that you can't go back to your previous level. Language learning is fascinating; once you understand something, you won't forget it.

Currently, in my day-to-day life as a freelancer, I spend a significant amount of time improving my English. I read a couple of chapters on Kindle, browse TikTok for short videos, watch TV shows and documentaries on Netflix, and explore various content on YouTube. I do whatever interests me at the moment. My only requirement for myself is that since I'm choosing content I enjoy, I should focus and understand every sentence carefully.

Here are some statistics: I've watched tens of thousands of TikTok videos, liked around ten thousand of them. I've watched a hundred documentaries and movies on Netflix. I've read over a hundred English blog articles. I've read 15 English books. Achieving this progress took time and concentration, mainly during the second half of 2019. On average, I spent around eight hours per day, five days a week. My main focus was on improving my listening and reading skills.

I believe it's essential not to force yourself to learn. Persistence can be painful. Find the content and methods that you enjoy the most, and stick with them. Watching international TikTok videos is effortless and doesn't require persistence. If you find certain materials difficult, look for easier ones to learn from. Once you've mastered the easy ones, move on to slightly more challenging content. Language is omnipresent, and there's an abundance of topics and expressions to explore.

To make progress, reflect on how you learned Chinese. It wasn't by studying grammar and doing exercises; you watched many martial arts movies and TV shows before coming to school. You engaged in simple conversations with your parents before starting school. Similarly, learning English should be approached in the same way.

Back in high school, even after learning English, our listening and reading skills were still poor. In exam-oriented learning, we might not have been efficient in our studies. Apart from exams, did we take the initiative to learn on our own? Probably not. In the past, learning resources were limited, with only textbooks and exercises like "New Concept English"and language learning tapes. In elementary school, we only occasionally listened to audio tapes in class. In middle school, we had around 15 minutes of listening practice before evening self-study.

Despite the hassle of accessing the internet, we now have abundant resources available for self-learning. The internet provides us with ample material for learning English.

Learning anything might be similar to learning English. Believe in the power of time and allow yourself to indulge in the simplest knowledge. If you want to learn physics, start with a series of basic physics experiments. If you want to learn programming, start with graphical programming. If you want to improve your drawing skills, start by drawing simple things. Turn learning into a game; even studying English with TikTok is genuinely fun and addictive.

When it becomes a game, our focus is intensified, and thinking becomes enjoyable. If you find yourself struggling to understand, search for easier materials to read. Read English children's books and watch animated films for children; they are excellent resources.

We should consider how we encourage high school and college students and those interested in learning English. After sharing these insights, I want to end with a memorable and slightly exaggerated statement: Learning English like a native is genuinely simple and enjoyable.

The Journey of Learning English

2020.08.16 This post was originally written in Chinese. It has been translated to English to facilitate further translations into other languages.

After struggling for many years, trying many different methods, and using countless apps, I finally found a way to truly love English—so much that I don't need to force myself anymore. I can simply enjoy it. Now I keep the habit of using English

regularly: reading English news, reading English books, watching American shows without subtitles, browsing English websites, reading and listening to large chunks of content. I've even had some real conversations with foreigners, and once helped a Dutch client remotely through English communication, earning some money by building them a website. Let me share this journey.

Back in high school, I found the English articles in textbooks dull. But after reading *Hackers and Painters*, I discovered its author Paul Graham. He had written many fascinating essays full of original ideas. I went to his personal website, printed out some essays, and read them. Whenever I didn't understand a word, I looked it up in a dictionary—an actual paper dictionary back then. Each lookup took half a minute to a minute, very tedious. But the essays were so interesting that I kept going. I even tried reading some English books like Orwell's *Animal Farm*, but I gave up quickly—the vocabulary was too overwhelming, and I lacked patience.

In college, I revisited Paul Graham's website and read dozens of his essays, this time with Chrome plugins that allowed quick word lookups. It was much faster. Doing programming also exposed me to foreign technical Q&A sites like Stack Overflow and projects on GitHub. But at that time, my vocabulary was limited to programming terms. Conversational English and reading bestsellers were still out of reach.

In 2016, I went to the U.S. With my broken English, I could manage simple conversations—asking for directions, chatting with Uber drivers. But I often didn't understand what was said, and it was embarrassing.

After that, I got busy with work and never really studied English systematically. I only occasionally read an article or looked up words. My listening and speaking stayed poor, and reading was always a struggle.

In May 2019, while working at an outsourcing company, I wanted to get projects from abroad. I filled in some details on Upwork, and surprisingly, a client from Amsterdam reached out. We agreed to build a website, and communicated on WhatsApp.

We even had a few voice calls, which were painfully awkward because my listening was terrible. I had to ask the client to speak very slowly and repeat constantly. But in the end, we completed the project, delivered the website, and I earned a few hundred euros. During that time, I told myself firmly: I must get good at English.

I pushed myself through *The Accidental President* on Kindle Mac, a book about Truman during WWII. It took me more than a month, and it was brutal—so many specialized terms, place names, and sophisticated vocabulary. I looked up word after word, painfully slowly. But when I finally finished, I overcame a mental barrier. I realized: if I can finish such a thick book, then normal essays and everyday written communication shouldn't be a problem.

I continued with more English books: *The Everything Store* (about Amazon), *To Pixar and Beyond*, *Jony Ive* (about Apple's designer), *Surely You're Joking, Mr. Feynman*, *Sapiens*, *The Meaning of It All*, the first ten chapters of *The Feynman Lectures on Physics*, *The Great Gatsby*, *A Mathematician's Apology*, *A Brief History of Time*, and more. Some I quit after a chapter or two.

Reading English books really trains patience. You have to empty your mind, enjoy "wasting time,"and accept the slowness. Never aim to finish—just read casually. If I kept thinking "I must finish this book,"I'd get anxious and unable to think deeply about each sentence. And if I didn't understand sentence by sentence, it would feel pointless. So instead, I adjusted my mindset: just flip through casually. Strangely, when I had nothing else to do, I'd keep reading anyway. Each book took me one to two weeks; in Chinese, I'd finish in one or two days. So patience was key.

Reading built vocabulary and comprehension, but not listening. For listening, I watched Netflix shows and documentaries without subtitles. I told myself: it's fine if I don't understand, just watch the visuals and catch words here and there. This way, there was no pressure. I ended up watching dozens of documentaries: *Mission Control*, *Our Planet*, *American Factory*, *Spielberg*, *The Bleeding Edge*, *The Dawn Wall*, *The Edge of Democracy*, *Hondros*, and a few seasons of *Friends*. People on Douban even made lists I followed.

After that, my listening improved somewhat, though I still missed many details.

I had tried vocabulary apps like Youdao, syncing words I looked up to review later. But I gave up. Memorizing words felt unnecessary. If I don't know a word, I can just look it up. Common words naturally stick after multiple encounters; rare words, even if memorized, would be forgotten by the next year. Why suffer? Even in Chinese, there are obscure characters I don't know, but it doesn't affect daily life.

The real breakthrough came with TikTok (international Douyin). I've praised it many times, but I'll say it again: it's the best English-learning tool.

Short, immersive, fun videos feel like face-to-face conversations with foreigners. A 10–60 second video can be rewatched endlessly. The infinite feed keeps your attention. Learning becomes play. With a high-school level of English, after two to three months of scrolling, you can understand most of it.

TikTok also opens your eyes to the world. The app recommends things you didn't even know existed. Unlike books, where you must first decide whether to read, TikTok shows you instantly, in vivid short videos. This combination of recommendation and short format is powerful.

Far from causing anxiety, TikTok calmed me. What causes anxiety is endless text feeds. Short videos, on the other hand, immerse you.

The best way to learn is to play—to not even realize you're learning, yet keep going. TikTok achieves that. On Chinese Douyin, we see everyday life across China. On international TikTok, we see the world.

Over the past year, I've averaged four hours a day on TikTok. Gradually, I found I could understand everything. Once you truly understand, you never forget. Learning a language feels magical. TikTok has top-quality content, short enough to avoid anxiety, and optional—watch if you want, stop if you don't.

Compared to reading books or watching long shows, which I struggled to stick with, TikTok suited my impatience. After trying almost every method, I finally found one that works for people like me: casual, fun, and effective.

I also tried joining English-learning groups on Telegram, chatting by voice with a friend from Iran for hours to practice speaking—basically a free language partner. He wasn't a native speaker, but his English was much better than mine, so I learned from him. We even had video calls, showing each other our living environments. Once, I also video-called an Indian guy who introduced me to his family, while I introduced mine. Incredible—making foreign friends from home.

Looking back, I see I wasted time forcing myself, instead of making learning fun. Like how I learned Chinese as a kid—immersed in martial arts TV dramas, I could understand and speak before school. Immersion and communication matter most. Today's internet makes it easy to immerse ourselves in top-quality English content without leaving home.

I believe once you master 1,000 basic words, you should focus on listening—watching TikTok, shows, or movies. Get extremely fluent with that core vocabulary instead of memorizing rare words. For elementary to high school students,

TikTok could be an amazing way to pick up English naturally while learning about the world.

Once those 1,000 words are second nature, interest and language sense will drive you to learn more.

Sometimes I think of taking TOEFL just to prove my ability. But then I realize—it's pointless. I'm not studying abroad or becoming a teacher. I know my own level. A certificate doesn't change whether I actually use English daily.

Now English is indispensable in my life: TikTok, YouTube, Netflix, Twitter, Google.

And yet, I also understand that living only with Chinese is fine too. Chinese books and knowledge are abundant enough. One can live well without English. So this essay isn't about creating anxiety, but sharing an easier method for those who want it.

Of course, TikTok requires some setup—VPN, SIM card tricks, a spare Android phone. I even taught a friend recently how to use it; it took 30 minutes.

This journey also made me reflect on learning itself. For me, anything that requires discipline and forced persistence won't work. Only play, joy, and genuine recognition keep me going. Only when I'd rather do this than anything else, can I truly learn.

English Animation Journey of My Daughter

{: .respon-
sive }

I have lived in Guangzhou, China in recent years. When I was working in Tianhe CBD in 2022, one day, I went to the office building and parked in a nearby parking spot that was part of a residential community. I saw a father using an iPad to help his son enter the elevator while quizzing him. His son answered in English. His son was quite young, like 4 or 5 years old. I was quite impressed at that time.

The houses there were worth 6 million or 10 million CNY at that time. They were quite expensive. So they were kind of rich. And they took care of the education of their kids so much. I had heard that people work hard to help their next generation succeed. I just hadn't seen the actual scene in real life.

One thing is that when I was in my 20 to 25, I didn't see many kids in my life. But when I had kids myself, I noticed that kids are everywhere.

Due to my recent years working as a contractor for foreign banks, my English improved a lot, and I use it at work or for personal study.

I take English learning as a priority for my daughter too. My daughter was born in 2021. In December 2023, I posted a video of my daughter singing "Baby Shark doo doo"and "Finger Family"while counting her fingers, and "Happy Birthday to You"on WeChat Video. This video gained more likes and views than my normal videos.

At that time, she was 2 years old. And I think she is quite inspiring for kids of similar ages in China.

Singing like that is not easy. It is not easy to access YouTube Kids in China. They are blocked by the Great Firewall.

So I bought an Apple TV and a Fire TV Stick. The inability to access them causes conflicts between me and my parents. My parents would like to give my daughter Chinese animations to watch. However, I prefer to give her English animations to watch.

And sometimes, if the VPN proxy for the Apple TV or Fire Stick doesn't work, then my parents are unable to fix it. My wife is unable to as well. And at this time, if my daughter still wants to watch English animations and I am not available, she starts to cry for it.

At first, my parents were not able to comprehend why I do this. They think Chinese is okay too. I know. But now, in this society, the competition is too high. She is growing up in China, so she will know Chinese surely. However, English education is what she is missing. I spent several years in my 20s learning this language. So if she is able to achieve that familiarity or get IELTS 7 before 20, then she is going to perform well in the international corporate world.

And there are lots of fights or arguments with my wife that I shouldn't let her use my second mobile phone—a Google Pixel—to watch English animations much. I think that's not a problem because I can reverse myopia. Please check out my paper.

It is really hard to change the opinions of others, including relatives. After seeing my daughter's progress in English, they started to accept letting her watch English animations.

There are people saying that mobile phones are killing people's dopamine systems. We shouldn't let kids touch digital devices at their early ages.

I don't think so, actually. What's the difference between English animations and an English environment? An English environment is just a physical and real English animation. The colorful scenes may stimulate brains. But how about the colorful playground?

Like Geoffrey Hinton and Elon Musk, who have been involved with digital technology for more than 4 decades, would digital devices change their brains? They are tools to record and share information. It depends on how you use them.

And we give our daughter English animations to watch while we go on a trip. I put my second mobile phone on the headrest of the seat, stuck in the empty gap, so it won't fall down.

As of June 2025, my daughter probably has 500 hours of English watching experience. Starting from April 2023 to now, two years have passed. It is like one hour a day. Though she is still not able to sing more English songs, we don't ask or practice this purposely. But surely her brain has changed a lot.

Through English animations, she knows a lot too. I don't know what she watches. If we ask, she can tell. And when we talk about something, she mentions that from some English animation too.

Think about it: if every kid in China starts to use this way, in less than 20 years, China will become like Singapore. Lee Kuan Yew turned Singapore into an English-speaking country within half a century.

The good news is that though there is the Great Firewall to separate China and the global world, almost all popular English videos on TikTok we can see on Chinese WeChat videos or Douyin too, with Chinese captions as well. There are tons of materials.

When I worked as a contractor in a Singaporean bank, I heard that in our team, some Indian colleagues speak English at home with their kids. It was quite astonishing to me. Think about China. People graduated from Tsinghua or Peking universities may not have that discipline or culture to speak English to their kids.

But I think for some parts of India and for Singapore, it is normal and natural.

My daughter is living in the future. Maybe one day all kids from all nations will watch English animations since they are two years old. And within a few generations, like in 2050, 80% of the population can speak English.

{: .responsive }

Parenting Tips

2025

- Safety is the most important thing. Live low-key and ensure that kindergartens or schools do the right thing.
- Broaden their perspectives by exposing them to diverse experiences and traveling.
- Avoid forcing them to do things they dislike and minimize their hardships.
- Encourage independence and freedom in decision-making.
- Allow flexible schedules for activities like sleep, especially if you have a flexible lifestyle.
- Foster a love for learning and curiosity; teach practical skills.
- Teach them to handle potentially dangerous situations safely without overprotecting.
- Equip them with practical skills, such as using tools and understanding technology.
- Foster confidence and fearlessness in learning and exploration.
- Teach them to respect others and be polite in interactions.

- Allow them to make their own decisions from an early age.
- Avoid criticizing; always encourage and provide positive reinforcement.
- Introduce them to modern technology and tools to prepare them for the future.
- Share your experiences and learnings with them, and be open to their opinions and ideas.
- Learn from them by observing them and understanding the differences between adults and kids.
- Provide YouTube Kids animations to watch and Apple Arcade and Google Play Pass games to play.
- Use YouTube Premium to download some English animations for kids to watch in the car.
- From 3 years old: Teach how to use a remote control, play mobile games, interact with ChatGPT via voice, and unpack deliveries.

A Practical Guide to Comfortable Daily Eyeglass Use at 1.50D Below Full Prescription

Last updated in 30 August 2025

Table of Contents

1. Reversing Myopia
    - Key Research and Resources
    - Todd Becker and Yin Wang's Methods
    - Related Articles and Papers
    - Multifocal and Progressive Lens Solutions
2. Natural Vision Restoration Tips
    - Understanding Myopia Causes
    - The "Just Barely Clear"Principle
    - Prescription Reduction Guidelines
    - Daily Usage Recommendations
    - Progressive Improvement Expectations
3. Natural Vision Restoration Progress
    - 3-Year Personal Experiment Data
    - Detailed Prescription History
    - Measurement Tracking Methods
    - Mobile-Optimized Progress Tables
4. Myopia: A Modern Yet Reversible Disease —Todd Becker
    - Foundation Research Videos
    - Todd Becker's Scientific Presentations
    - Historical Context and Discovery
5. It's Easy to Make Mistakes Without Hands-On

- Responding to Critics and Skeptics
- Importance of Experimental Evidence
- Personal Family History Analysis
- Hands-On vs Theoretical Approaches
- Corporate World Parallels

Reversing Myopia

Last updated in August 2025

- Myopia: A Modern Yet Reversible Disease, M.S. (AHS14), Todd Becker
- Myopia: a modern yet reversible disease, Todd Becker
- Experimental Verification of Yin Wang's Natural Vision Restoration Method, Zhiwei Li
- The Discussion Regarding Astigmatism When the Eyeballs Reverse to the Normal Shape, Zhiwei Li
- Natural Vision Restoration: The Principle of "Just Barely Clear", Zhiwei Li
- Natural Vision Restoration Tips, Zhiwei Li
- Using Multifocal Progressive Lenses to Reverse Myopia, Zhiwei Li

---

Natural Vision Restoration Tips

2024.11.29

- The reason people worsen their eyesight is that they use glasses with full prescription (which lets them see a blackboard clearly at 5m) to look at phones, computers, or books most of the time. This causes their eyeballs to deform.
- After using glasses with a 150-200 degree reduction for several days, you won't want to go back to full prescription glasses. The latter becomes very uncomfortable.
- Understanding myopia is key - it's caused by eye muscle changes, not just genetics or light exposure.
- The most important principle is "just barely clear"- wear glasses that make things just clear enough without straining.
- For computer or phone use, a 150-200 degree reduction often works well.
- Keep a stronger prescription pair for driving and essential tasks requiring perfect vision.
- Expect slow progress - about 25 degrees reduction every three months is good.
- Don't aim for perfect clarity. Slightly blurry vision is part of the improvement process.
- Use the formula: Myopia degree = 100/clear distance (in meters) for rough assessment.
- Don't force yourself to see clearly - let your eyes adjust naturally.

- Learn about your eye mechanics - understanding how external eye muscles work helps make better vision choices.
- Myopia requires wearing glasses, which can be troublesome during activities like playing basketball or when wearing a hat. The hat cannot stretch to cover more skin and provide warmth because the glasses take up space. Forgetting to wear glasses also affects vision.
- While some people may feel too old or lazy to address their myopia, it's crucial to take action for the sake of future generations to prevent the same challenges.
- The improvement process takes time - myopia developed over years will take time to improve.
- The point is that most of the time, our eyes can clearly see the objects we want to view. If we move objects about 10 cm farther away, they become unclear. This indicates that we are using our eyes correctly.
- Since we use phones or computers most of the time, we need to reduce the prescription strength of our eyeglasses to use our eyes correctly.
- Consider the case of presbyopia lenses. As people age, their eye muscles change shape in a direction opposite to that in myopia. We can apply the principles used in presbyopia lenses to myopia lenses.
- I recommend multifocal progressive lenses. Based on your prescription, assuming correct vision is 0.9, the suggestion is to make the additional correction for near vision +1.75 D in myopia (with astigmatism remaining unchanged), while the additional correction for far vision is +1.00 D in myopia (with astigmatism remaining unchanged).

Natural Vision Restoration Progress

Last updated: November 28, 2024

My Eye Prescription History

Whole Number Format

Left Eye

Time	Myopia	Astigmatism	Vision
2022.03.05	350	225	1.0
2022.11.13	325	200	0.9
2023.04.20	300	125	1.0
2024.01.07	300	100	0.9
2024.11.28	250	100	0.9
2025.08.30	300	100	0.9

Right Eye

Time	Myopia	Astigmatism	Vision
2022.03.05	575	175	
2022.11.13	550	175	
2023.04.20	500	125	
2024.01.07	475	125	
2024.11.28	425	125	
2025.08.30	500	100	0.9

Decimal Format

Left Eye

Time	Myopia	Astigmatism	Vision
2022.03.05	-3.50	-2.25	1.0
2022.11.13	-3.25	-2.00	0.9
2023.04.20	-3.00	-1.25	1.0
2024.01.07	-3.00	-1.00	0.9
2024.11.28	-2.50	-1.00	0.9
2025.08.30	-3.00	-1.00	0.9

Right Eye

Time	Myopia	Astigmatism	Vision
2022.03.05	-5.75	-1.75	
2022.11.13	-5.50	-1.75	
2023.04.20	-5.00	-1.25	
2024.01.07	-4.75	-1.25	
2024.11.28	-4.25	-1.25	
2025.08.30	-5.00	-1.00	0.9

Myopia: A Modern Yet Reversible Disease —Todd Becker

I discovered Todd Becker's video in the footnotes of Yin Wang's article on the Natural Vision Improvement Method. The video, uploaded in 2014, has garnered over 1.2 million views.

Myopia: A Modern Yet Reversible Disease —Todd Becker, M.S. (AHS14)

How to Reverse Myopia With Todd Becker (Ancestral Health Today Episode 021)

It's Easy to Make Mistakes Without Hands-On

2025.07.11

After Yin Wang proposed the myopia reversal method in his blog, there have been some opposing views on this. I happened to read some views.

Since 2022, I have been conducting the experiment just as this paper describes.

It is easy for me to know what is right or wrong. It is incorrect to say that heredity is the primary risk factor for myopia. In Todd Becker's YouTube Video, in the introduction, there is a paragraph:

> Myopia, or near-sightedness, is generally assumed to be an irreversible, genetically determined condition that can only be ameliorated with corrective lenses or surgery. Its prevalence is 30-40% in the U.S. and Europe, and more than 50% in some Asian countries, but it is rare in Africa and in pre-industrial cultures. The incidence of myopia correlates with IQ, school achievement, and industrialization, suggesting that an environmental factor is at work—namely, near-work.

From my experience, I can say this is correct. My father has 3 brothers. Three out of the 4 siblings don't have myopia, while one does. The youngest brother of my father has myopia and has the highest education, probably a bachelor's degree. They were born in the 1960s and 1970s.

For myself, I started developing myopia when I was in junior high school around 2009. At that time, I probably had around -2.00 diopters. I recall that I stayed up at night and used my Nokia phone while lying in bed with the lights off. After a summer like that, I felt that my eyesight had become poor.

I happen to know some of the authors of opposing views. They also blog or comment. After I conducted a 3-year experiment, and Yin Wang tried so many ways to reverse myopia, the first thought when I see such opposing views on the Internet is that we don't care. They don't know how much effort we have made, how we rejected our ideas or started again, how we adjusted our experiments, or how we mulled over all these things.

The author has a good heart in warning us about Yin Wang's discovery. But really, the author has not yet conducted experiments on it. I agree with a lot of his ideas about startups, as he has a 10-year startup journey. But for myopia itself, most of our efforts in experiments or reading—and most human discoveries or discussions about it—don't matter. Only a few truths matter.

We are not disrespectful towards the opposing views. Science is science. It doesn't matter that there are 2 billion people with myopia. And a large number of them use eyeglasses with full prescription that allow them to see a blackboard 5m away, but they use them to look at mobile phones a lot. That's why their myopia becomes deeper and deeper.

One easy way to know whether what Yin Wang says is true is to just put down your glasses. If your myopia is just 175 or 200 degrees, you can see your mobile phone without needing glasses. And if you are wearing eyeglasses with full prescription, you can put the mobile phone far away, like 80 cm or 1 meter, and you probably can still see the words on the screen.

I can see that one of the big mistakes is that they don't do hands-on experiments while we do a lot and think about them extensively. They just say what the book says.

In the corporate world, I can see this scenario too. In a team, engineers and leaders may come up with solutions. However, half of them probably don't work. We need to try the solutions to see what happens. It is just like the tens of thousands of times I thought that code should work but testing failed in my 10 years of software engineering experience.

From the hands-on aspect, we actually can do a lot of filtering. We should hire, communicate, and work with those people who are more hands-on. In the company, those who like hands-on work will probably be better leaders. Steve Jobs once highlighted the importance of highly performant individual contributors.

So when judging the quality or truthfulness of opinions, we can check their hands-on degree. Does the author do a lot of experiments?

And I can see that people who do a lot of hands-on experiments actually don't hold their opinions too rigidly. I know there is still doubt in reversing myopia. What degree is best for reversing myopia, whether 150 degrees less, or 200 degrees less? I wrote a paper about this, Natural Vision Restoration: The Principle of "Just Barely Clear".

It is really easy to talk, but harder to perform actual experiments, work out a theory, and then test again, just like writing code.

It's easy to make mistakes without hands-on experience. After making tons of mistakes, you will realize that compared to the losses or pain that the mistakes cause us, the trouble of doing a lot of hands-on experiments to get to the truth and avoid mistakes is negligible. That said, like in stock investing, one day you will know that compared to the losses or pain that investing mistakes cause us, the trouble of performing hands-on, thorough experimental efforts to invest successfully is negligible.

The Journey of Reversing Myopia Naturally

2025.01.05

Besides three papers about reversing myopia, I want to share more about this discovery story.

> Li, Zhiwei. (Jun 2023). Experimental Verification of the Natural Vision Restoration Method. Zhiwei's Blog. Web. PDF.
>
> Li, Zhiwei. (Jun 2023). Astigmatism in Reversing Myopia. Zhiwei's Blog. Web. PDF.
>
> Li, Zhiwei. (Nov 2024). Natural Vision Restoration: The Principle of 'Just Barely Clear'. Zhiwei's Blog. Web. PDF.

Patience

As reversing myopia naturally is kind of slow, just like losing weight. In my 3 years of experience (from February 2022 to January 2025), I achieved the following:

Time	Left Eye Myopia	Left Eye Astigmatism	Right Eye Myopia	Right Eye Astigmatism	Correct Vision
2022.03.05	-3.50	-2.25	-5.75	-1.75	1.0
2022.11.13	-3.25	-2.00	-5.50	-1.75	0.9

Time	Left Eye Myopia	Left Eye Astigmatism	Right Eye Myopia	Right Eye Astigmatism	Correct Vision
2023.04.20	-3.00	-1.25	-5.00	-1.25	1.0
2024.01.07	-3.00	-1.00	-4.75	-1.25	0.9
2024.11.28	-2.50	-1.00	-4.25	-1.25	0.9

Let's use Python to draw a paragraph about it.

img

Figure 5: img

So, you need to be patient about it.

Do the Right Thing, Results Will Happen

As long as you do the right thing, results will happen. Just like losing weight, eating very little and playing sports often will help you lose weight. If you lose 1 kg every month, then one year you will lose 12 kg.

Similarly, in learning, if we study 10 hours a day, then one year it amounts to 3,650 hours.

The good news for reversing myopia is that there is nothing hard about it that you cannot persist with. It just requires you to see a mobile phone or computer screen barely clearly.

This is a better and more comfortable way to proceed. The cost of using the correct method is that I need to prepare another pair of eyeglasses with full prescriptions to drive, see views while traveling, and watch movies in theaters.

Those times in life are actually just a small part, like at most 2 hours in a day, with the remaining 15 hours being awake.

Don't Settle, Continue to Improve

In the first year, my primary activity involved computer work, during which I wore glasses with a 1.50 diopter reduction. This setup allowed me to frequently experience the "just barely clear"state, where vision was sufficiently clear without being overly sharp. However, in the second year, as I shifted my focus to studying for my associate degree at home, I reduced my computer usage and primarily relied on my mobile phone for reading materials and exercises.

After talking with a friend, I started to learn about bifocal lenses and multifocal progressive lenses. I even learned terms like segment height, segment position, and working distance for vision.

It required a lot of experimentation. Grasping the principle, the solutions can vary—from a plain +1.50D addition to the prescription, to a +2.00D addition, and then using multifocal progressive lenses to help. There are still ways to improve.

After ChatGPT came out, there have been many improvements. A lot of smart people are thinking in that direction. In the real world, there are many use cases and more real problems to solve.

For reversing myopia, it is exactly the same. Older people use presbyopia lenses.

Share to the Urgently Needy

Though I shared my reversing myopia article on the WeChat Subscription Platform in June 2023, in December 2024, I received a reply from someone who claimed his 12-year-old kid was frustrated with his myopia degree of 500, that is, -5.00. When I was 12, I thought I had just begun to wear glasses for myopia with a degree of -2.00.

As the truth or discovery is less known to people, only those in urgent need want to try this. This reversing myopia method, probably first introduced by Todd Becker in 2014, will become popular one day.

There are no negative effects of the new adjustment method. Additionally, more and more people will develop myopia, and their myopia degrees will become larger and larger.

Does a Conspiracy Exist in the Popularity of Myopia

People wearing full degrees of eyeglasses have caused their myopia to deepen. The myopia isn't made by someone deliberately. It is just that human beings are not careful. It seems normal to wear full degrees to regain normal eyesight of 5.0, the eyesight of a child.

However, what human beings don't know is that they use them to look at near objects. The people who invented multifocal progressive lenses or vision testers knew a lot about how the eye or myopia works.

So, the improvement here is just that we categorize how we use our eyes. We observe that human beings nowadays are mostly using phones or computers. And it doesn't matter that we see things far away vaguely in daily life.

The point is that most of the time, our eyes can clearly see the objects we want to view. If we move objects about 10 cm farther away, they become unclear. This indicates that we are using our eyes correctly.

How to Find New Discoveries Again

Try to observe and learn more on the Internet. Be careful of minor opinions. Try experimenting by yourself.

There are a lot of improvements in life. After the truth was found, it seems obvious. People just haven't thought about it before. Steve Jobs surely understood it. Steve Jobs"Think Different"campaign is good.

More Confident to Improve the World

With so many people wearing glasses, whenever I see people wearing them - whether offline or online - I recognize there's room for improvement in this area.

Their eyeglasses are too strong for viewing mobile phones or computers, which will cause their myopia to progressively worsen.

Complete Life Philosophy

Table of Contents

1. Life Tips
    - Exploring the World and Expanding Perspectives

    - Critical Thinking and Authority Questioning
    - Human Relations and Social Wisdom
    - Health and Natural Living
    - Freedom, Security, and Personal Growth
2. Productivity Tips
    - Focus and Mental State Management
    - Learning Through Movement and Activity
    - Natural Learning Methods
    - Flow State and Genuine Interest
    - AI-Assisted Learning Strategies
3. Learning Tips
    - Learning as Hobby and Playful Exploration
    - Practice, Repetition, and Hands-On Experience
    - AI Tools Integration (ChatGPT, Claude, Gemini)
    - Multi-Modal Learning (Video, Audio, Text)
    - Teaching and Content Creation for Learning
    - Historical Context and Principle-Based Learning
4. Money Spending Tips
    - Free Resources and Smart Shopping
    - Quality vs Price Analysis
    - Long-term Value Thinking
    - Learning-Oriented Purchasing
    - Strategic Financial Decisions
5. Innovation Tips
    - Understanding Innovation as Trying New Things
    - Innovation in Different Contexts
    - Overcoming Resistance to Change
    - Building Innovation Through Experience
    - Critical Evaluation of New Approaches

Life Tips

- Explore the world early, go to different countries, and talk to different people.
- Read many books, especially non-fiction ones.
- Always think of better ways to do things.
- Think like a genius, suspect everything, especially authorities.
- Treat people equally. Be kind. Don't look down on people who are no better than you in terms of grades or salary. They didn't lie to you, trick you, or steal your money.
- Be careful of the powerful people behind the government. Some may hurt you from behind.

- If something happens, usually someone wants it to happen. Think hard about things that happen coincidentally or intentionally.
- Think hard about what is good or bad, and how everything gets invented or developed.
- Think about how to live in this world in the best ways.
- Get a lot of things that are free, such as free apps, books, and information. Enjoy air, sun, scenes, and water.
- Think hard about the rules. Figure out the original intention behind them. Ask yourself if the rules could be better or more detailed. Sometimes, we have to follow the rules. But do not let them limit your thinking.
- Treat everything as a learning experience. If it is good, it is nice. If it is bad, avoid it next time.
- Pursue truth. But be careful with science. Not all science is really true.
- Be safe. Health is the most important thing. Our body is really amazing.
- Eat natural or fresh things as much as possible. Eat fruits instead of snacks. Modern society invents too many foods. Prefer natural or less processed ones. Live long and explore more.
- Don't pursue miracles. Life itself is a miracle.
- Don't live for recognition or fame. We don't need recognition when we are born. We need security. Parents punish us when they don't approve of our behaviors. So gradually, we need recognition to get security. But those two are different things.
- It's okay to do anything as long as it doesn't involve or hurt anyone else.
- It's better to learn the truth first. So we don't need to change later.
- English is the world language now. Be good at it and use it like a native language.
- You don't need to change anyone. If you discover something or create something that people want, they will come.
- Don't get desperate. The majority of the world is helping. They are making products or providing services for cheap or free.
- When it comes to money, think about how the richest or smartest people do it.
- Normally, you can't force anyone to do anything, and others can't force you to do anything either.
- Follow your true passion. You can't force yourself to do things you don't like for a long time.
- You are upset that something doesn't work as expected. Just accept the facts or change your worldview to adapt to it. Learn from it.
- The good welfare of you depends yourself, your family, your company, your country. Comparing them wisely in the world. If they are all good, you will have good life.
- Almost everything can be changed. Think about your nationality. Think about where you were born. Remove every limit and start thinking about how to have the best life in the world. Start to notice the big gap and work towards closing it.

- Humans are born to learn. If every piece of knowledge is taught correctly or you explore and reflect on it often, you will be happy to gain it.
- Think often about what Yin Wang or Daniel P. Friedman says. They are probably the smartest and kindest people in the world.
- The world will have more freedom. The world will become better as knowledge and technology evolve.
- Focus on learning Science, Technology, Engineering, and Mathematics (STEM). These fields are generally based on reproducible and verifiable facts, unlike some other areas of knowledge. Prioritize learning correct information initially to avoid needing to correct your understanding or habits later.
- Learning AI/STEM is better than learning additional natural languages, as they are a more essential way to understand and interact with the world.
- Your perception of your salary matters more than the amount itself. If you compare it to others, it will likely never feel sufficient. Also, how you spend your salary matters; if you buy what you need daily at a low cost, you might feel it is adequate. For example, using it to buy food or a mobile phone might feel more justifiable, but using your salary to buy a house might not seem as worthwhile.
- Try a lot of products or services. Try a lot of options, sizes, brands, and varieties. People work hard to manufacture or provide them. There are so many free or cheap valuable things.
- When dealing with car insurance, ensure repairs are done as inexpensively as possible to keep next year's fees from increasing significantly. Although the insurance company currently covers the cost, it will ultimately affect your premiums. Once, my headlight was slightly damaged, but they replaced it with a new one. Next time, avoid doing this.

Productivity Tips

- Focus at work doesn't always happen as you might expect. Surprisingly, noisy or interrupted environments, like when babysitting children, can sometimes boost productivity. Your mood and mental state significantly influence your ability to concentrate, much like how they affect sleep quality.
- The main dilemma of the world is that knowledge and invention are increasing, however, our focus and study are becoming harder. Most days, it's challenging to sit down and read books or watch courses. You can't force yourself to focus, just like you can't force yourself to fall asleep.
- To address this, I like to walk and drive a lot while listening to lectures or watching videos. Covering 10,000 kilometers and walking 5,000 steps every day has helped me become fluent in English and pass the IELTS exam. Now, I hope it can also help me learn math and AI. The key is to avoid mobile phone distractions and remain calm. I encourage others to try this method and provide me with feedback.
- Have fun and work on your own projects often.
- Experiment with different approaches to find what comes naturally to you. Pay attention to what actually works in practice rather than wishful thinking. Trust your instincts and physical responses as reliable indicators.

- The ways people read, write, or code vary significantly. Personally, I often prefer printing papers to read them.
- Less is more. Smaller, focused teams in companies often outperform larger teams, even when the larger teams have 10x or 100x more people, due to better communication and coordination.
- I read parts of “The Mythical Man-Month”by Fred Brooks. Its principles about software project management remain relevant in the long term.
- No productivity method works automatically - you need to experiment and adapt techniques to find what works best for your specific needs.
- Use urges to do things. Don’t force yourself. Always do what you want at the moment. This is fulfilling.
- Use code or other tools to automate tasks, allowing you to focus on the most interesting aspects.
- When you truly enjoy working or learning, you don’t worry about progress, and you don’t have a goal to rush towards. You simply want to do or learn things for their own sake. You don’t worry about how much time has passed or the difficulty of the materials. You enter a flow state.
- Sometimes, it’s hard to focus—even just reading a tutorial on the screen can be difficult. However, when you’re writing code and get stuck on a bug, you may find yourself fully focused. Try to find ways to write and run code as part of your learning process.
- There are several ways to learn something. Get hands-on and use AI to help you learn or conduct experiments. You could also work through a book or a series of course videos in a fixed order, or learn by taking exams or answering questions. Basically, you can learn with a plan or learn by following your mood. Learning can be curiosity-driven or success-driven. Always choose to learn by mood.

Learning Tips

- Treat learning as a hobby by approaching with playfulness and genuine interest.
- Learn through practice and repetition until mastery comes through hands-on experience.
- Use daily immersion by engaging with the subject regularly in natural contexts.
- Follow your genuine interests because passion leads to better learning outcomes.
- Draw from multiple sources whenever you need simpler explanations.
- Persist through difficulties by taking breaks and seeking inspiration from peers.
- Maintain consistent engagement instead of relying on cramming sessions.
- Create opportunities to actively apply what you’re learning.
- Make learning practical by connecting it to real-life applications.
- Remember that learning comes naturally through play and exploration.
- Learn with AI tools, ChatGPT, Claude and Gemini.

- Use multiple languages to ask AI tools questions.
- Use YouTube, YouTube shorts or TikTok to search for video explanations.
- Download YouTube courses to listen to while driving, try to recall or imagine what you listen to.
- Emotions come and go - when you can't focus, try using other ways to learn, or do something else, then come back.
- It is ok to not fully understand a question or paragraph in one go - just skip it and try to understand it next time, or the time after that.
- In school, I learned by listening to teachers and doing exercises. After leaving school, I learned by doing work, projects and pursuing hobbies. Reflect on how you learned things in the past. Use those methods to learn current matters.
- Action produces information. Just move forward to learn new things.
- Take exams, interviews, certificates, finish tasks to achieve excellence.
- Do not just watch videos of the subjects pretending to understand.
- Most of the time you think you have already prepared well, but actually haven't. Aim high to achieve middle results. Aim for middle to achieve low results.
- Sleep well. Take naps if tired.
- Attitude or motivation is hard to lie to yourself about. Follow your true passion.
- Environment is important. You don't need to learn English hard if you grow up in English-speaking countries.
- Your brain is powerful. Think things through by yourself. Think again and again, time after time.
- Grab the essence or principle of things to learn. For example, machine learning is about calculus, and startups are about making something people want.
- You don't need to learn something to prove yourself to others. New discoveries take time to become popular. Your abilities take time to become truly good. Others will then notice.
- Learning is mainly about trying new things or thinking in new ways. Keep exploring.
- To learn is to better enjoy life. For example, learning a new language is like opening a new world.
- When learning a subject, directly approaching the core concepts or doing the final exam exercises is okay. You can learn backward to the fundamental things. A hard start leads to an easy end. Quickly overcome the fear or discomfort associated with new knowledge.
- Teach or share what you learn to others, one piece at a time. No matter how technical or sophisticated it is, share it with your companion, friends, or even street buddies.
- Write blogs, make videos, or create other content to share. People are narcissistic. By reviewing what you create, you will notice imperfections. That will give you the urge to improve.
- Learn the history of things, like some specific math, electricity, or programming. It shows how things develop and the strong reasons for the order in which they appear. These parts are what textbooks miss.

- Often, if you learn a topic and encounter some difficulty, just repeat watching a video, listening to it, or reading materials about it around 20 times—it can have a magical effect. As June Huh said, repetition has some beauty.

Money Spending Tips

- Find free solutions first - open source software, free ebooks, papers, information etc.
- Prefer affordable items. Many things under 100 CNY are quite good quality. You're less likely to suffer a loss or be cheated.
- I bought an iPhone with 512GB storage, but 256GB would have been enough. Photos and videos take up half my space. There are solutions like syncing to computer or using iCloud. Usually, a moderate solution works fine.
- Instead of buying things just to use, buy things to learn from. Regularly buy new affordable items that you can learn from.
- Carefully observe what people are using. Be less influenced by media.
- I bought some mechanical tools, phone repair tools, and maintenance tools. They're quite useful - I can learn from them and use them occasionally.
- Protect your belongings. Use covers to protect your computer or iPhone.
- If something is inexpensive, buy several units, like charging adapters or electrical cables.
- You learn what's good or bad by using things, especially over long periods. Keep an eye on your items - for example, you might need to change a MacBook battery after 5-8 years.
- Don't buy things just to show off, like houses or cars. The price of vanity keeps getting higher.
- For learning, the key is to focus your mind on the subject rather than expensive materials or courses. Most things can be learned for free.
- Analyze why you don't use expensive items you've bought. Often it's due to lack of time, actual need, or genuine interest.
- Focus on essence rather than appearance. Drinks in pouches are often cheaper than boxed drinks. Compare prices per 100ml for the same type of juice - pouch drinks are usually more economical.
- Learn more details to make better purchases.
- The first purchase is important, especially for expensive items requiring bank loans. Start with cheaper options to try - like with housing.
- Consider different solutions: when playing basketball with glasses, you could buy sports goggles, use a strap to secure regular glasses, or keep spare pairs. Think about long-term costs and effectiveness. Apply this thinking to other purchasing decisions.
- Purchase components like batteries or other parts from an online store and visit a local repair shop for installation, such as replacing the battery in a mobile phone or laptop.

Innovation Tips

- Innovation means trying new things.
- The value of innovation is amazing. It doesn't matter if there are a million or a billion people. Sometimes, nobody can think of something new.
- In a group of 10 or 20 engineers, some individuals are able to think in new ways, using new tools or different approaches to help the project. I admire them.
- In the current world, this AI era, the power of innovation is becoming more and more important.
- AI is innovation. The "Attention Is All You Need"paper is innovation. A different way to query a database is innovation. The ways that surrounding peers seldom do or think about that actually help are innovations. Innovation is everywhere.
- Sometimes, new ways of doing things are innovations for me, for a group of people, for a small startup, for big companies, for a large country, or for human beings.
- People don't inherently hate innovation, but rather hate innovation that will hurt their interests. Some of Nokia's employees hated Apple's innovation. In the 1600s, influential figures within the Catholic Church hated Galileo's heliocentric innovation.
- Before innovators try new things, they have already tried a lot of old things, and someday, somehow, they come up with new ideas.
- While the majority might notice a few innovations from some people, those individuals have usually already made a lot of innovations.
- Trying new things doesn't always guarantee success. They sometimes show a decline instead of progress. Be critical and observant.

Investing

- Use a dedicated bank account for P2P trading when buying cryptocurrency in China. This makes it easier to record videos or take screenshots, allowing merchants to verify the transaction history, as the transaction history is short.
- Prefer bank transfers over WeChatPay or Alipay, as they are less regulated. If one bank account is not working for transfers to merchants, use another one.
- If we borrow consumer debt from big platforms, we should deposit it directly into the bank card that we will use for P2P trading. The merchants on Binance need to verify the source of the money.
- It's often preferable to hold the native coin of a network; for example, SOL on the Solana network or ETH on the Ethereum network. These native coins are typically used to pay for transaction fees and are widely accepted across mainstream cryptocurrency exchanges and decentralized exchange (DEX) platforms.

- Avoid using cross-chain bridges due to their high costs. For example, I once paid a $100 USD network fee to convert $1200 USD worth of SOL to ETH. Instead, transfer your crypto to an exchange that supports the same chain, and then use the spot market to convert.
- Do not hold USDT. Instead, hold ETH, SOL, and BTC. Change the traditional mindset. Measure everything from the perspective of the crypto, rather than measuring crypto coins from the perspective of legal tender.
- Personally, I use Binance, OKX, and Phantom.
- Consider what can be trusted and what cannot.
- Most things are assets and have values measured in money. Over time, the value of some assets will increase, while the value of others will decrease. Exchange less valuable assets for more valuable ones.
- The significance of anyone's thoughts or actions should be relative to their market capitalization. Because I have few holdings, my thoughts are of little consequence to the overall market.
- Analyze things objectively and consider how others act. Think about things long term.
- Morality matters less in trading. Actions allowed by law are acceptable. Sometimes, the law is outdated. If you trust something, invest money, and then lose it, you just need to accept it.
- Them: Congrats! You made a lot of money on Bitcoin, right? Changpeng Zhao: No, not exactly. I still have the same number of Bitcoins. It's just that everything else has become a little cheaper relative to Bitcoin.
- In regular investing, the key is to invest consistently over the years. Therefore, there is no need to rush and attempt to invest more in any given week or month, especially if the money is not from your savings but is borrowed, has a deadline, or has an associated cost. It can be difficult to maintain consistent investing over a year or a few years.
- The P&L of your portfolio may be zero after regularly investing for months. So why not invest all at once at the start? The feeling is different. You have been riding up and down with these stocks. You gain resilience and patience, which is priceless.
- Regular investing is very interesting. The price of stocks or coins goes up and down. I observe how the market works. Often, people debate or react to news in emotional ways. But I understand a lot about what I invest in. Every time I question its long-term value, I haven't changed much about my strategy.
- Regular investing gives you a good feeling when you see the prices of stocks come back. Prices go up and down, and you know time will tell.
- Don't borrow money to invest; save money instead. Regular investing takes years to generate profit, and anything can happen in the market. Money that is due soon won't help you invest for the long term. If a bad situation arises and you have to liquidate your investments to repay the money, you will probably incur a significant loss.
- The key to regular investing is accumulating a significant volume of investments and identifying the right trends. It is better to invest more capital during the later phases. Avoid increasing investments during bull markets, as a downturn may follow in the subsequent months, which would undermine the principle of regular investing. If you continue to invest during downturns, whether they last months or years, you will achieve a more favorable average entry price.

- Borrowing money to invest has another problem: it generates interest. For example, a 10% yearly interest rate means you need to gain at least 5% on your investment just to cover the expense. This makes investing harder, especially during downturns.
- The amount of money you put into regular investments should be such that you can neglect it without looking at it for days or weeks. Even if it loses more than 50 percent, you can accept it. With this kind of mindset, you can invest for the long term and make profits.
- If you care too much about the performance of your regular investing portfolio and frequently open your stocks or crypto app, it means you have invested too much money, borrowed money, or are not confident in your long-term strategy.
- Big money waits.
- Don't chase bull markets or follow the crowd. If you invest regularly, it's better to increase contributions during bear markets. In the long run, stocks will likely rise significantly, but short-term bubbles may still occur.